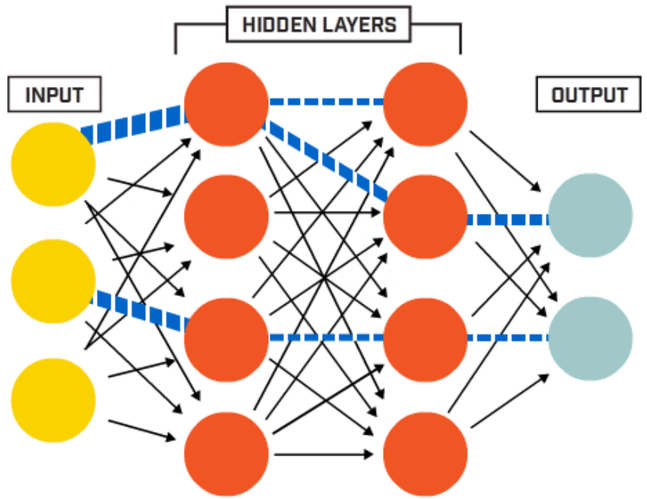
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

DEEP LEARNING BASICS



7.1 Biological Motivation

7.1.1 Structure and Function of Biological Neurons

7.1.2 How Neurons Communicate: Synapses

7.2 Neural Network Representation

7.2.1 Basic Structure of an Artificial Neuron

7.2.2 Activation Functions

7.2.3 Layers and Connections

7.3 Problems for Neural Network Learning

7.3.1 Classification Tasks

7.3.2 Regression Problems

7.3.3 Pattern Recognition

7.3.4 Time-Series Prediction

7.4 Perceptron and Multi-layer Networks

7.4.1 Structure of Perceptron

7.4.2 Learning algorithm for Perceptron

7.4.3 Single-layer and Multi-layer Perceptron

- **Neural Networks**
  - Brain – complex, non linear and parallel computer - capability of organize its structural constituents called neurons so as to perform tasks like pattern recognition, perception, motor control etc
  - A Neural network is a massively parallel distributed processor made up of simple processing units which has a natural propensity for storing experiential knowledge and making it available for use. It resembles human brain in two aspects
    - Knowledge is acquired by the network from its environment through a learning process
    - Interneuron connection strengths known as synaptic weights are used to store the acquired knowledge
  - Procedure to perform the learning process is called learning algorithm

- **Benefits of Neural Networks**
  - Neural Networks achieve power through massively Parallel and Distributed Structure
  - Generalization – refers to produce reasonable outputs for inputs not encountered in training
- **Properties of Neural Networks**
  - Non Linearity -- Neurons are distributed throughout the network eg speech signal
  - Input – Output Mapping -- Reordering of previously trained examples and mapping Input – output using estimation statistics
  - Adaptivity -- Stability Plasticity Dilemma
  - Evidential Response -- Decision and Confidence , rejection of unwanted patterns
  - Contextual Information – Every neuron is affected by the activity of global neurons
  - Fault Tolerance -- hardware is fault tolerant, robust in nature , performance degrades in adverse conditions
  - VLSI Implementability -- well suited due to massive parallelism
  - Uniformity of Analysis and Design -- commonality, share and integrity
  - Neurobiological Analogy

- **Human Brain**

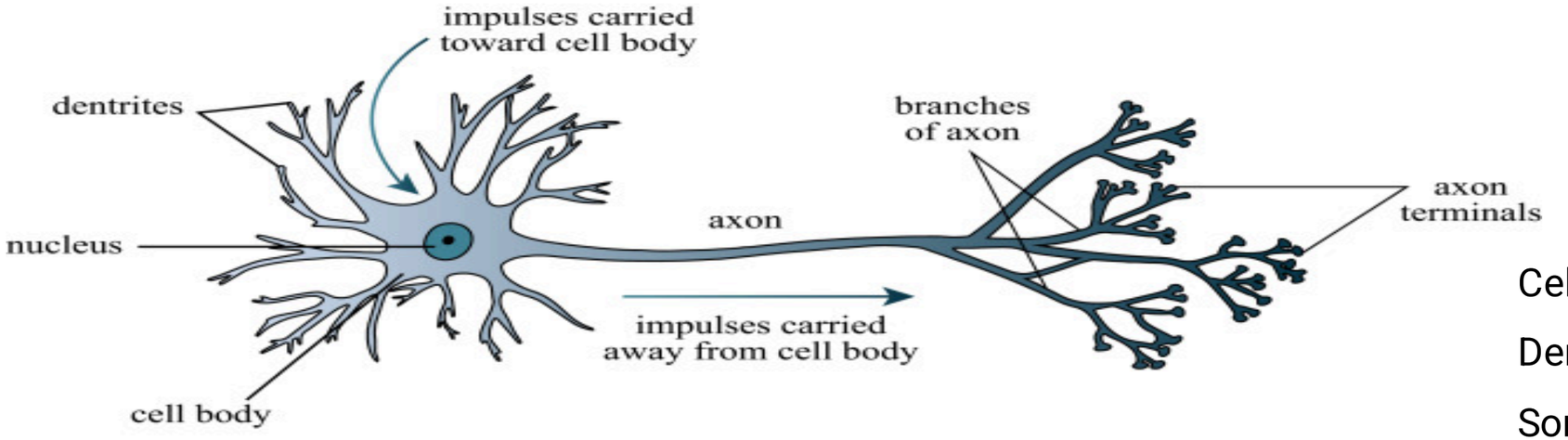
The **human brain** contains about 10 billion nerve cells, or neurons. On average, each neuron is connected to other neurons through approximately 10,000 synapses.

	processing elements	element size	energy use	processing speed	style of computation	fault tolerant	learns	intelligent, conscious
	10^{14} synapses	10^{-6} m	30 W	100 Hz	parallel, distributed	yes	yes	usually
	10^{8} transistors	10^{-6} m	30 W (CPU)	10^{9} Hz	serial, centralized	no	a little	not (yet)

Unit -4

Characteristics of BNN and ANN

Characteristics	Biological Neural Network	Artificial Neural Network
Speed	Processes information at a slower rate. Response time is measured in milliseconds.	Information is processed at a faster rate. The response time is measured in nanoseconds.
Processing	Massively parallel processing.	Serial processing.
Size & Complexity	An extremely intricate and dense network of linked neurons of the order of 1011 neurons and 1015 interconnections.	Size and complexity are reduced. It is incapable of performing sophisticated pattern recognition tasks.
Storage	An extremely intricate and dense network of linked neurons with 1015 interconnections, including neurons on the order of 1011.	The term "replaceable information storage" refers to the practice of replacing fresh data with old data.
Fault tolerance	The fact that information storage is flexible means that new information may be added by altering the connectivity strengths without deleting existing information.	Intolerant of faults. In the event of a system failure, corrupt data cannot be recovered.
Control Mechanism	There is no unique control mechanism outside of the computational task.	Controlling computer activity is handled by a control unit.

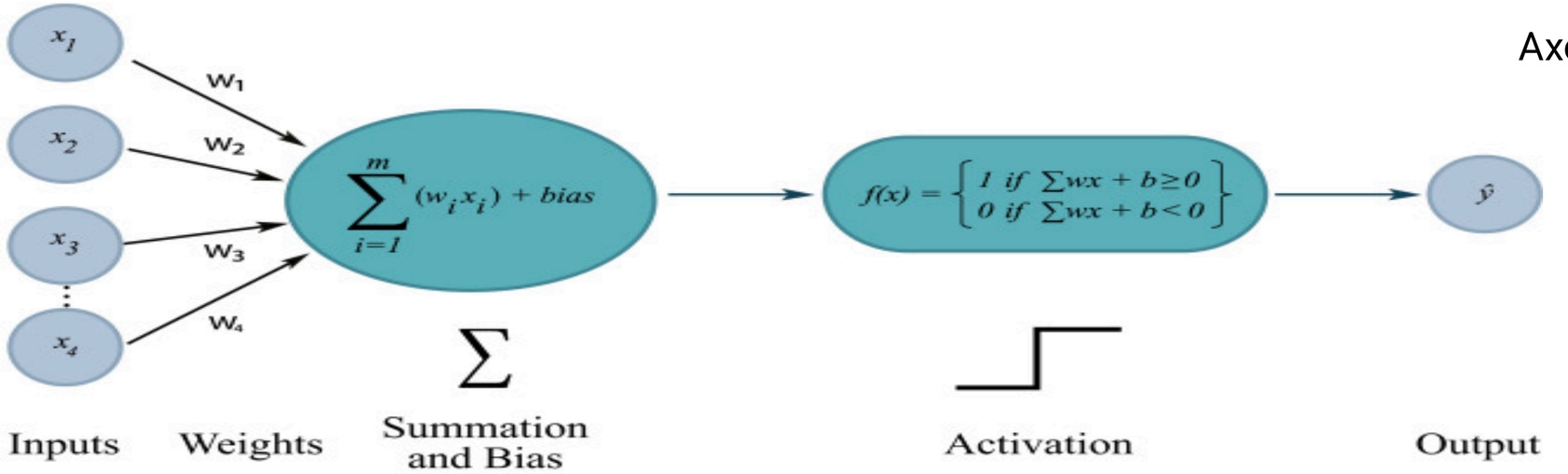


Cell – Neuron

Dendrites –Weight or interconnections

Soma – Net Input

Axon –Output



Models of a Neuron

Synapses are the connecting junction between axon and dendrites. The majority of synapses send signals from the axon of a neuron to the dendrite of another neuron

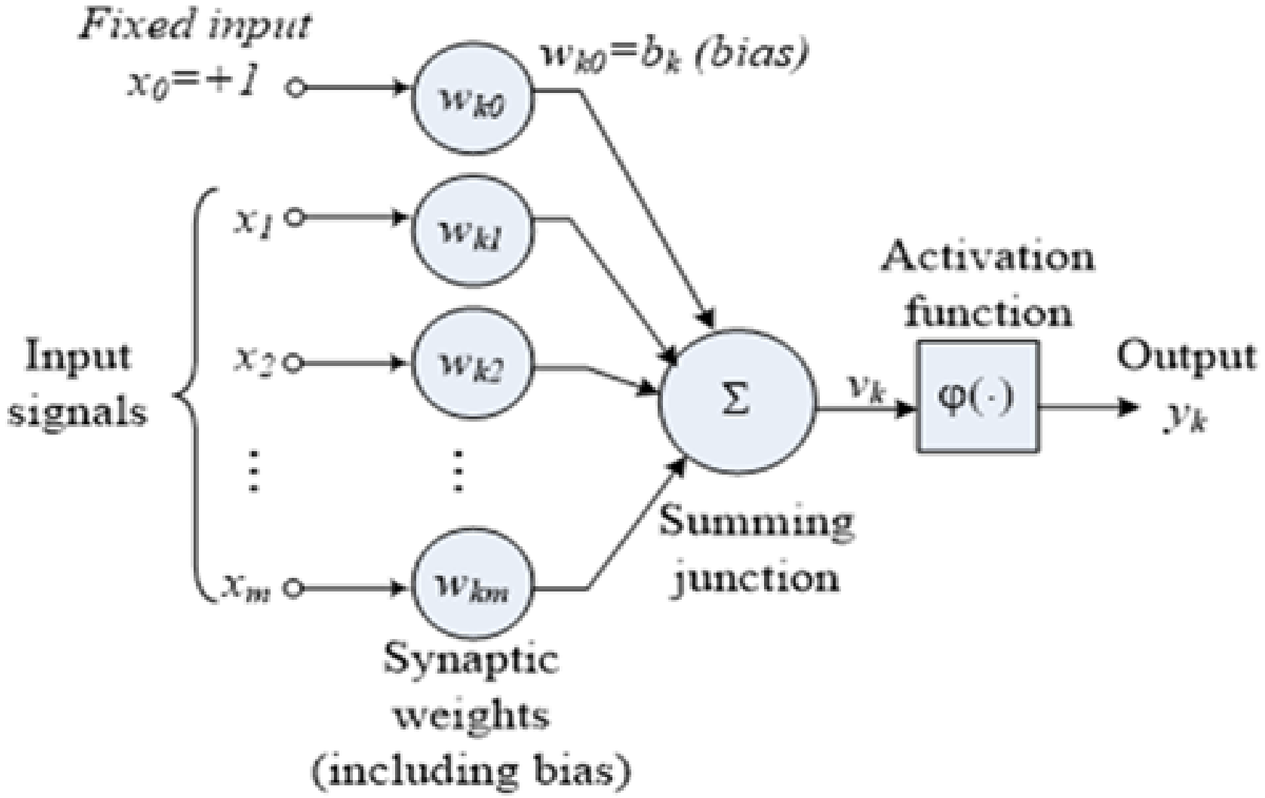
Dendrites have fibers branching out from the soma in a bushy network around the nerve cell.

- Dendrites allow the cell to receive signals from connected neighboring neurons and each dendrite is able to perform multiplication by that dendrite's weight value

Axons are the single, long fibers extending from the main soma.

- The axon will branch and connect to other dendrites

Non Linear Model of a Neuron



Functional Parameters of a Neuron

Function parameter	Description
w	Vector of real-valued weights on the connections
w · **x**	Dot product ($\sum_{i=1}^{n} w_i x_i$)
n	Number of inputs to the perceptron
b	The bias term (input value does not affect its value; shifts decision boundary away from origin)

Functional Parameters of a Neuron

Weight : Each Neuron is connected to other by means of communication link and each link is associated with weight.

- Weights contain information about the input signal that is used by the net to solve a problem.
- Represented in the form of a matrix and called as connection matrix.

Bias : Included in the network and has its impact in calculating the net input.

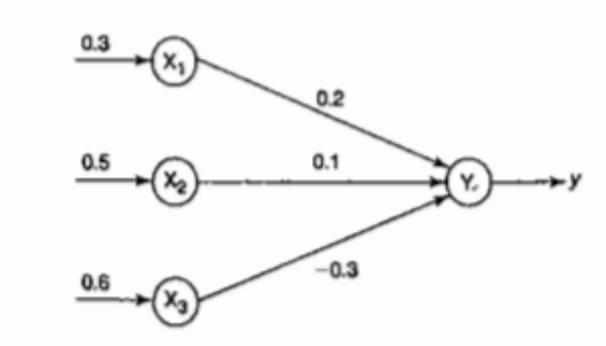
- Two types – Positive and Negative Bias
- Positive – helps to increase the net input
- Negative – Decreases the net input

Functional Parameters of a Neuron

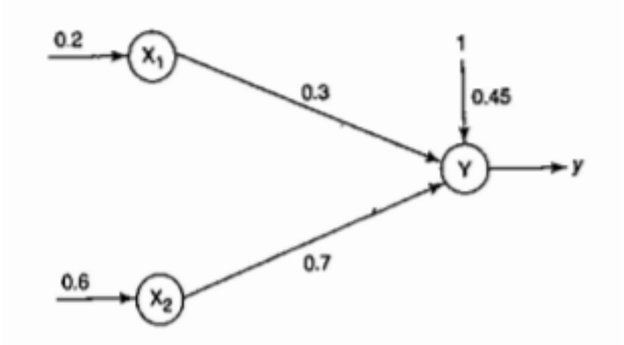
- Threshold : Is a set value based upon which the final output of the network is calculated.
  - Used in activation function
- Learning Rate : Denoted by ∞ . Used to control the amount of weight adjustment at each step of training
  - Ranges from 0 to 1
- Momentum Factor : Convergence is made faster if this is used.
  - Weights from one or more previous training patterns will be saved

Problems

- For the given network, calculate the net input to the output neuron



- Calculate the net input (i) Without bias and (ii) when the bias is 0.45 and observe the difference and discuss



Activation Function is a mathematical function used in artificial neural networks to determine whether a neuron should be activated or not.

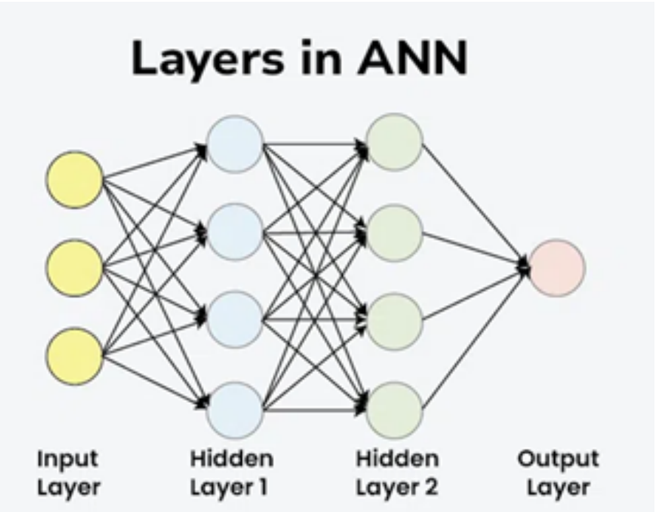
- **Add Non-linearity:** Helps neural networks model complex data like images, speech, and text.
- **Control Output Range:** Maps input values into specific ranges (e.g., 0–1 or –1–1).
- **Enable Learning:** Allows gradient-based optimization during backpropagation.

Type	Formula / Definition	Output Range	Key Points
Linear	Identity mapping	(-∞, ∞)	Used in regression; not suitable for hidden layers
Sigmoid	Maps values between 0 and 1	(0, 1)	Used in binary classification; can cause vanishing gradient
Tanh	(-1, 1)	Zero-centered; smoother than sigmoid	
ReLU	0 for x<0, x for x≥0	[0, ∞)	Fast and popular; can cause “dying ReLU” problem
Leaky ReLU	(-∞, ∞)	Allows small negative gradient; avoids dying ReLU	
Softmax	(0, 1), sum=1	Converts logits into probabilities for multi-class classification	

Layers and Connections

Input Layer

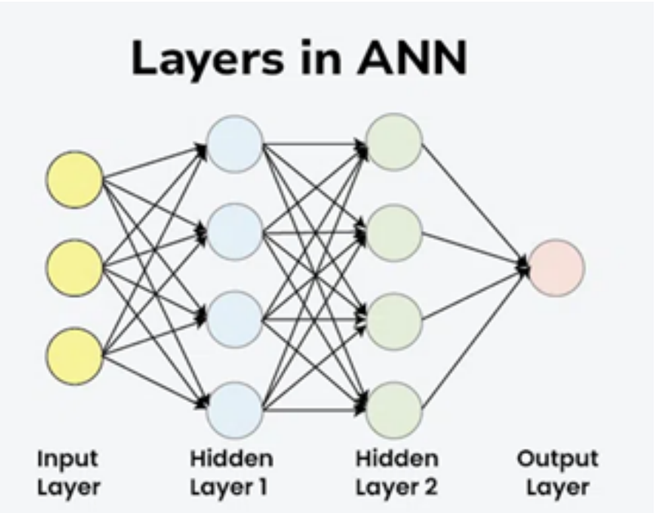
- Input layer is the first layer in an ANN and is responsible for receiving the raw input data.
- This layer's neurons correspond to the features in the input data.
- For example, in image processing, each neuron might represent a pixel value.
- The input layer doesn't perform any computations but passes the data to the next layer.
- Role: Receives raw data.
- Function: Passes data to the hidden layers.



Layers and Connections

Hidden Layer

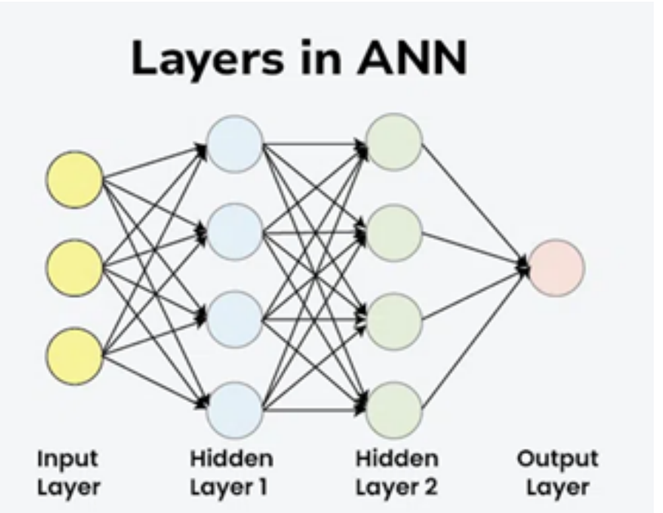
- Hidden Layers are the intermediate layers between the input and output layers.
- They perform most of the computations required by the network.
- Hidden layers can vary in number and size, depending on the complexity of the task.
- Each hidden layer applies a set of weights and biases to the input data, followed by an activation function to introduce non-linearity.



Layers and Connections

Output Layer

- Output Layer is the final layer in an ANN.
- It produces the output predictions.
- The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.
- The activation function used in the output layer depends on the type of problem



Layers and Connections

Types of Hidden Layer

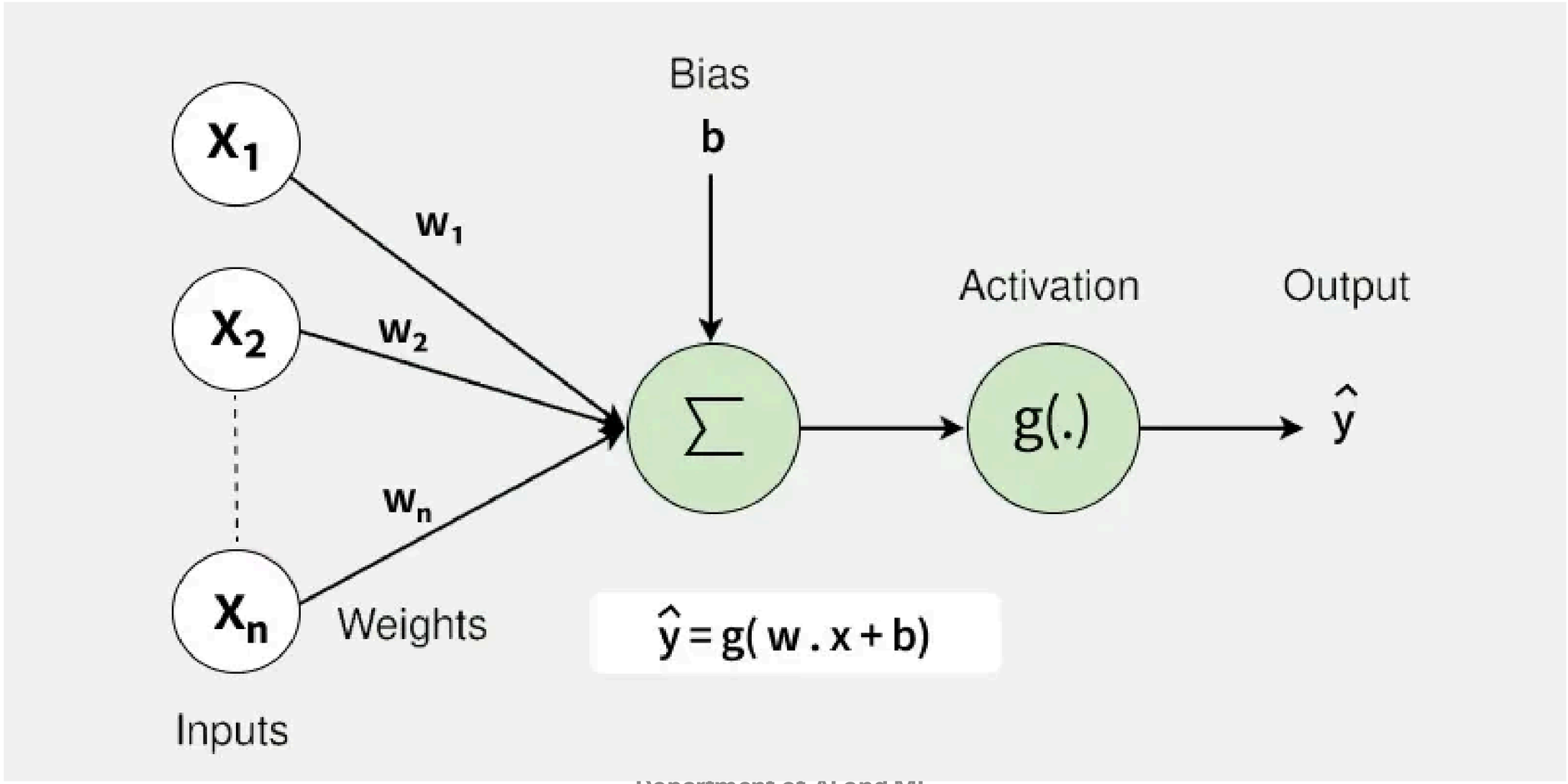
Hidden Layer Type	Description / Function	Purpose	Common Applications
1. Dense (Fully Connected) Layer	Every neuron connects to all neurons in the previous and next layer.	Learns global patterns and relationships.	Feedforward Neural Networks (FNN), MLPs
2. Convolutional Layer (Conv Layer)	Applies filters (kernels) to extract local spatial features.	Detects edges, textures, and spatial hierarchies.	CNNs for image, video, and audio processing
3. Pooling Layer	Reduces spatial dimensions by summarizing nearby outputs (e.g., max, average).	Decreases computation, prevents overfitting.	CNNs (e.g., VGG, ResNet)
4. Recurrent Layer (RNN, LSTM, GRU)	Includes feedback connections allowing information to persist.	Captures temporal/sequential dependencies.	RNNs, LSTMs for NLP, time-series data
5. Normalization Layer	Normalizes inputs across mini-batches or features.	Stabilizes training, prevents gradient issues.	CNNs, RNNs, Transformers
6. Dropout Layer	Randomly disables a fraction of neurons during training.	Prevents overfitting, improves generalization.	DNNs, CNNs, RNNs
7. Flatten Layer	Converts multidimensional data (e.g., 2D images) into 1D vectors.	Prepares data for fully connected layers.	Transition between CNNs and dense layers

Layers and Connections

Connection Type	Description	Example / Application
Feedforward Connections	Information moves only forward (input → hidden → output).	Standard Neural Networks
Recurrent Connections	Output of neurons feeds back into the same or previous layers.	RNNs, LSTMs
Skip / Residual Connections	Directly connect non-adjacent layers to improve gradient flow.	ResNet, Transformers
Convolutional Connections	Local connections using shared weights (filters).	CNNs
Fully Connected (Dense) Connections	Every neuron in one layer connects to every neuron in the next.	MLPs (Multilayer Perceptrons)

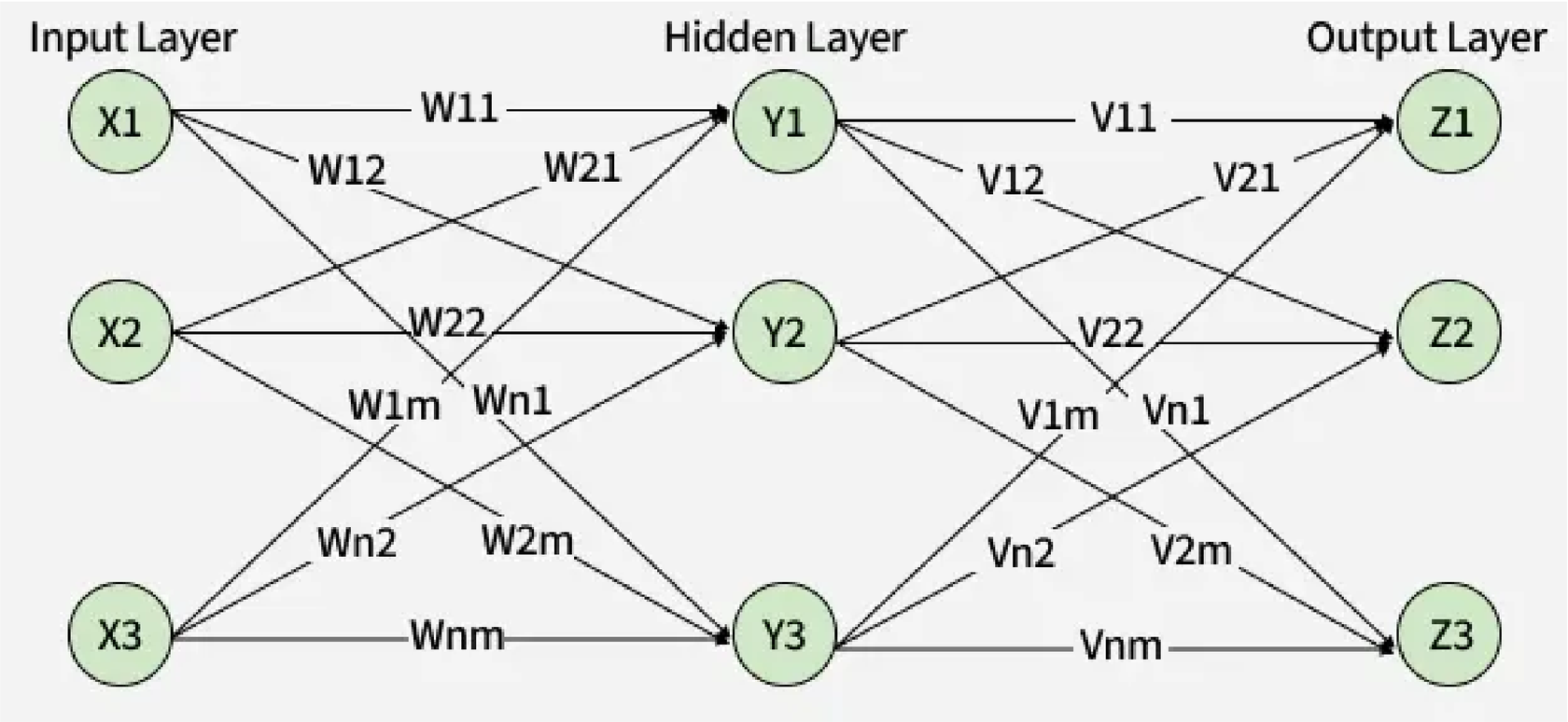
Types of Neural Network Architectures

Basic Structure



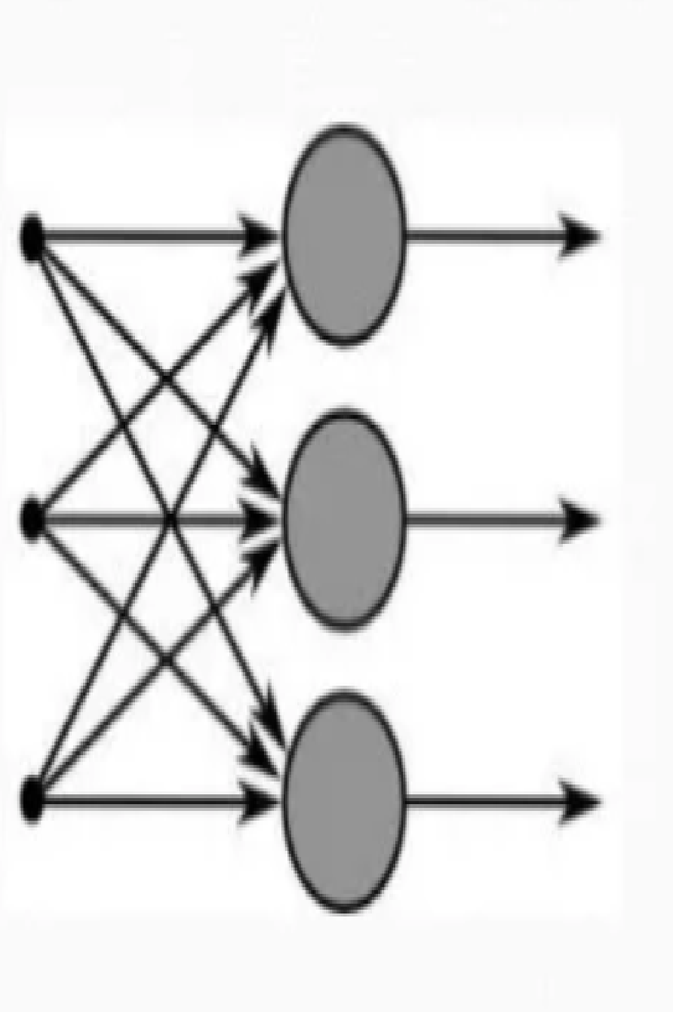
Types of Neural Network Architectures

Feed Forward Neural Network



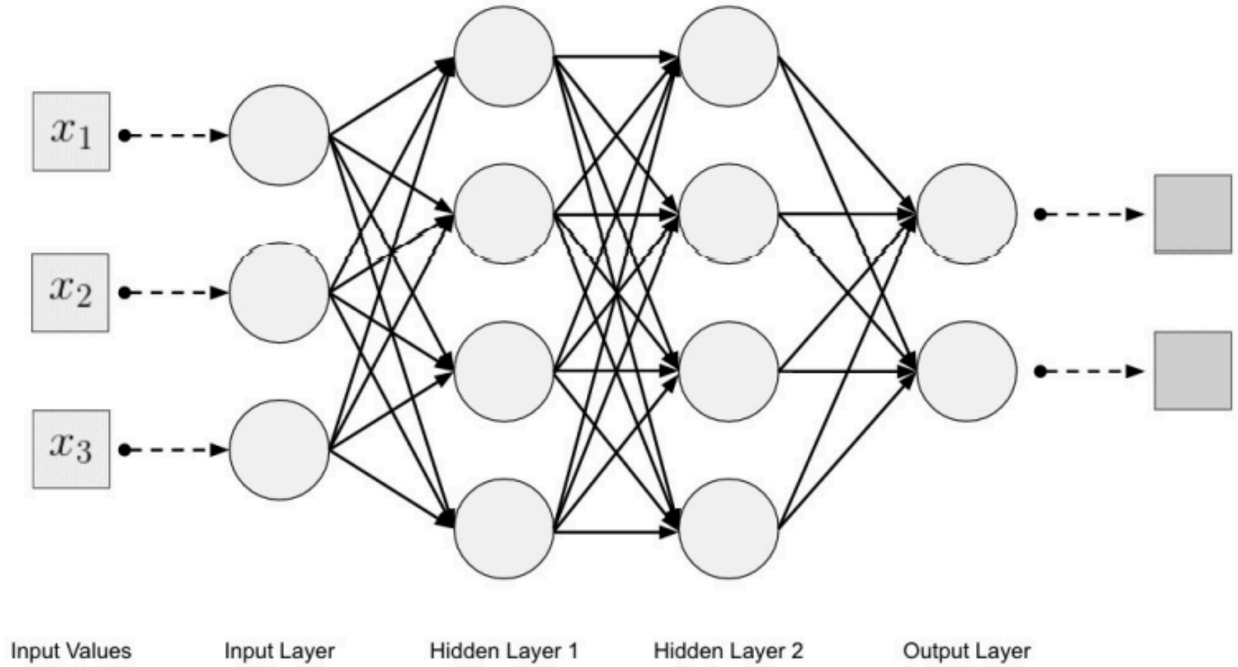
Network Architeures

- Single Layer Feedforward network
  - Consists of input layer of source nodes that projects to output layer of computation node
  - Strictly feedforward or acylic type
  - Single output layer of computation nodes
- Layer is formed by taking a processing element and combining it with other processing elements
- Interconnections lead to the formation of various network architectures



Network Architeures

- Multi Layer Feedforward network



Perceptron

- The Perceptron is the simplest type of artificial neuron, introduced by Frank Rosenblatt(1958).
- It is a binary classifier that decides whether an input belongs to one class or another based on a linear combination of inputs and weights.
- Perceptron consists of a single layer of input nodes that are fully connected to a layer of output nodes.
- This foundational model has played a crucial role in the development of more advanced neural networks and machine learning algorithms.

Types of Perceptron

Single-Layer Perceptron is a type of perceptron is limited to learning linearly separable patterns.

- It is effective for tasks where the data can be divided into distinct categories through a straight line.
- While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear.

Multi-Layer Perceptron possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

Components of Perceptron

Input Features: The perceptron takes multiple input features, each representing a characteristic of the input data.

Weights: Each input feature is assigned a weight that determines its influence on the output. These weights are adjusted during training to find the optimal values.

Summation Function: The perceptron calculates the weighted sum of its inputs, combining them with their respective weights.

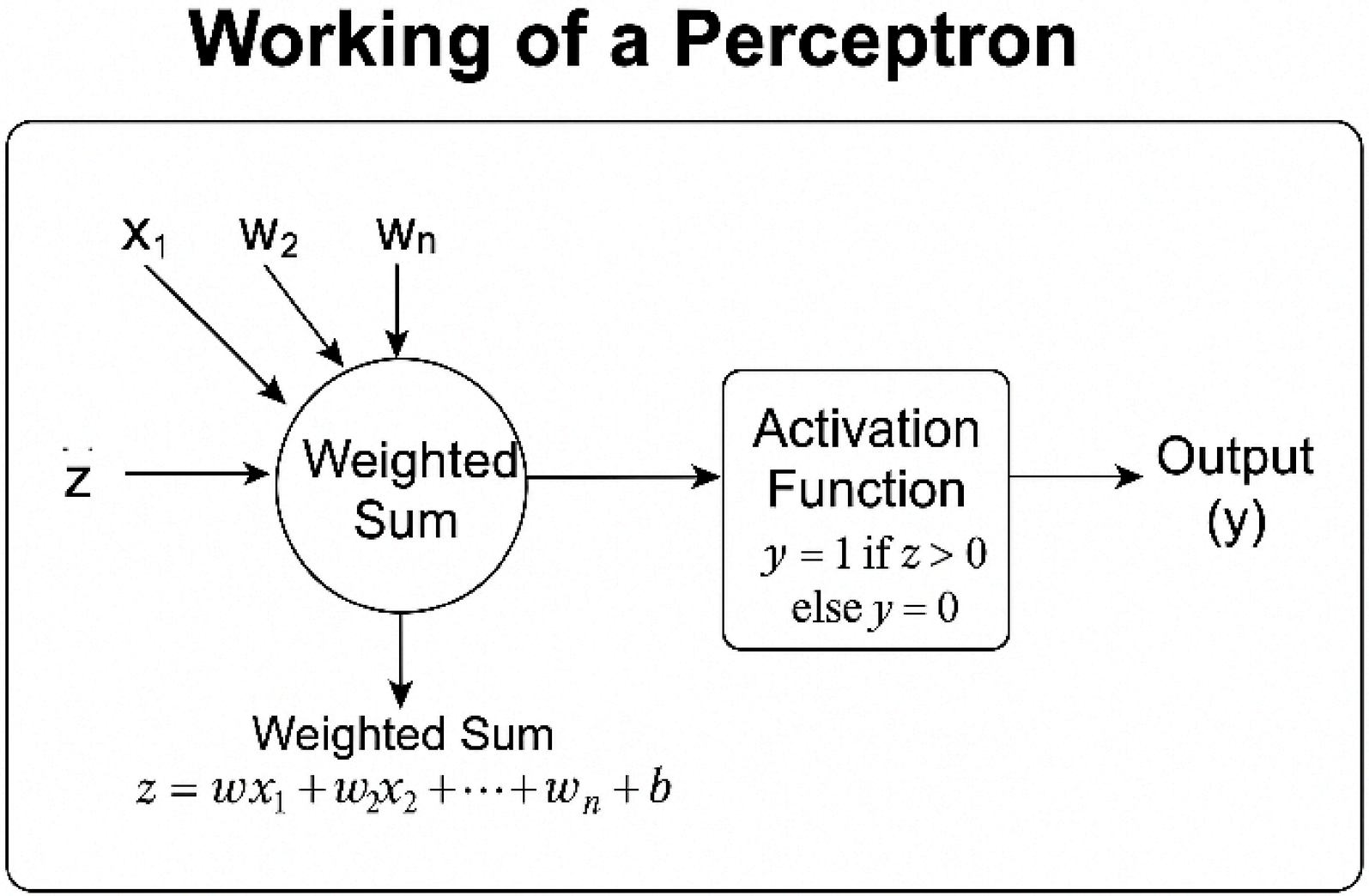
Activation Function: The weighted sum is passed through the Heaviside step function, comparing it to a threshold to produce a binary output (0 or 1).

Output: The final output is determined by the activation function, often used for binary classification tasks.

Bias: The bias term helps the perceptron make adjustments independent of the input, improving its flexibility in learning.

Learning Algorithm: The perceptron adjusts its weights and bias using a learning algorithm, such as the Perceptron Learning Rule, to minimize prediction errors

Working of Perceptron



Working of Perceptron

Step	Process	Explanation
1. Input Signal	(x_1, x_2, x_3, ..., x_n)	Each input represents a feature (e.g., pixel value, sensor reading, etc.).
2. Weight Assignment	(w_1, w_2, w_3, ..., w_n)	Each input is assigned a weight that represents its importance.
3. Weighted Sum Calculation	(z = w_1x_1 + w_2x_2 + ... + w_nx_n + b)	The perceptron computes the weighted sum of all inputs and adds a bias term (b).
4. Activation Function (Decision Making)	(y = f(z))	The result is passed through an activation (usually a step) function to decide the output.
5. Output Generation	(y = 1) if (z > 0), else (y = 0)	Produces a binary output: 1 (activated) or 0 (not activated).

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Gradient Descent Algorithm

Basic PPT with Fundamental Equations

Gradient Descent is often used as black-box tools

- Gradient descent is popular algorithm to perform optimization of deep learning.
  - Many Deep Learning library contains various gradient descent algorithms.
    - Example : Keras, Chainer, Tensorflow…

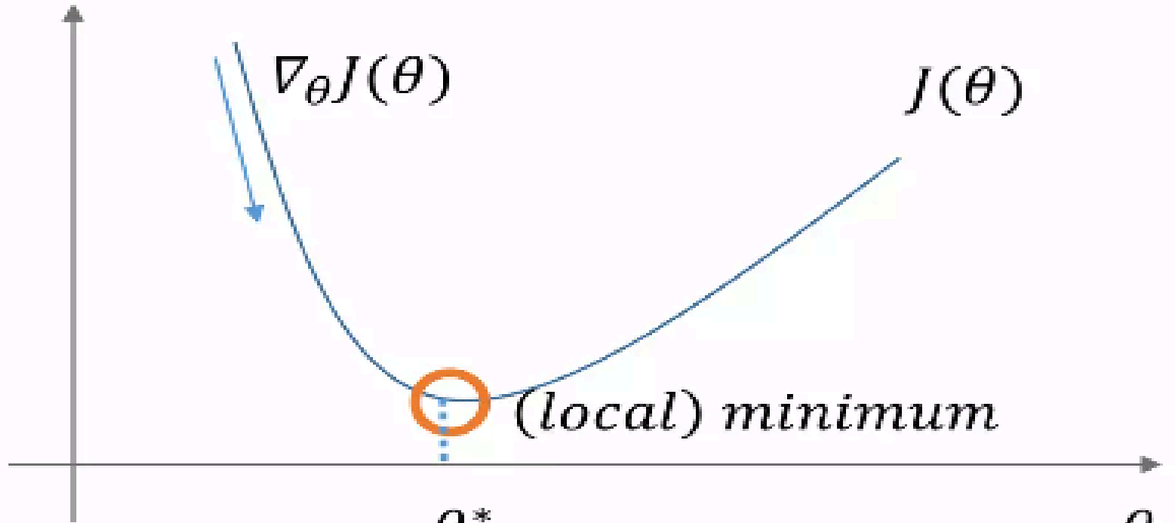
- However, **these algorithms often used as black-box tools and many people don’t understand their strength and weakness**.
  - We will learn this.

Gradient Descent

- Gradient descent is a way to minimize an objective function $J(\theta)$
  - $J(\theta)$: Objective function
  - $\theta \in R^d$: Model's parameters
  - η : Learning rate. This determines the size of the steps we take to reach a (local) minimum.

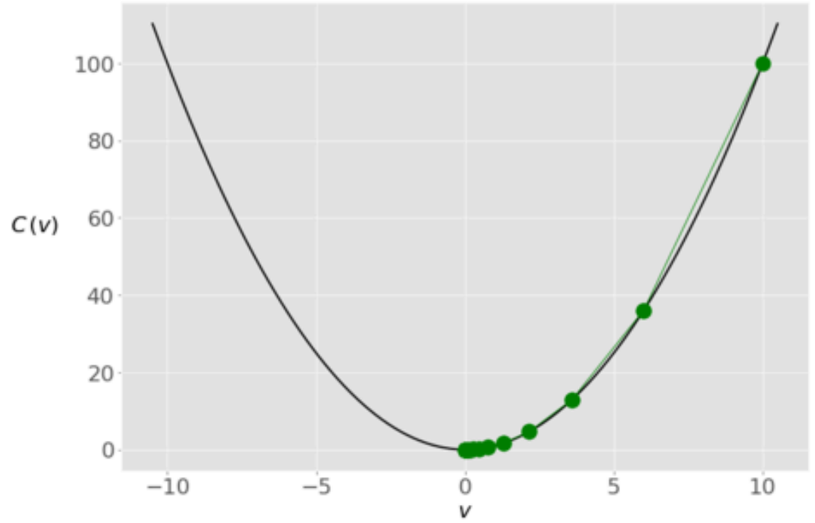
Update equation

$$\theta = \theta - \eta * \nabla_\theta J(\theta)$$



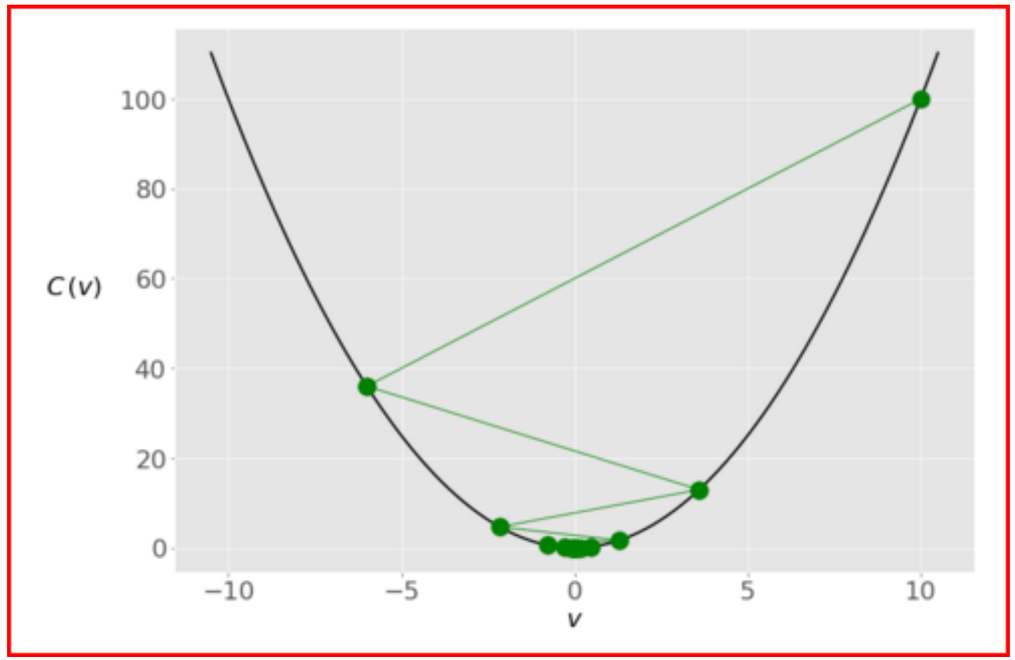
value `10.0` and set the learning rate to `0.2`. You get a result that's very close to zero, which is the correct minimum.

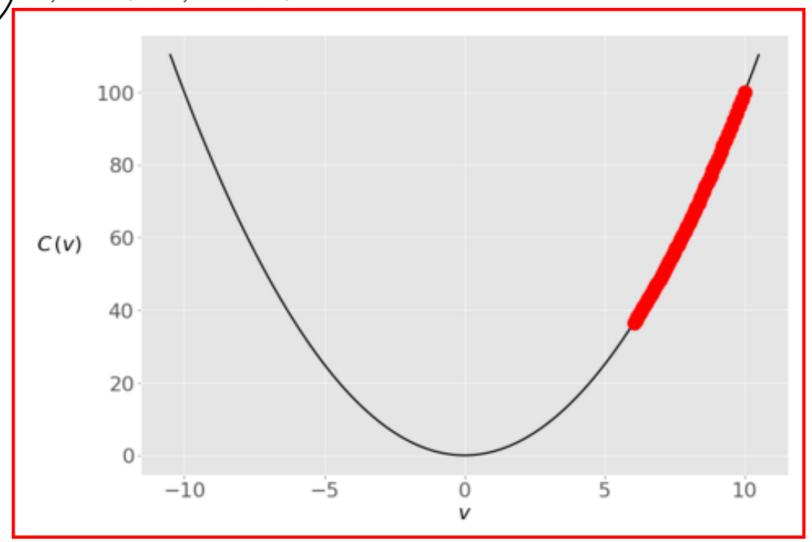
The figure below shows the movement of the solution through the iterations:



Learning rate 0.6

Learning rate 0.8





Learning rate 0.005

Recall – Comparison between GD and SGD:

1. For the big data analysis, the performance of GD is slow , this is where need of SGD has came .

1. SGD is similar to GD except , it only looks for one sample per step. One sample can be picked randomly

1. SGD is used when the redundancy in the data set is found.

1. Mini batch SGD , we take k data point among n data points given

Trade-off

- Depending on the amount of data, they make a trade-off :
  - The **accuracy** of the parameter update
  - The **time** it takes to perform an update.

Method	Accuracy	Time	Memory Usage	Online Learning
Batch gradient descent	○	Slow	High	×
Stochastic gradient descent	△	High	Low	○
Mini-batch gradient descent	○	Midium	Midium	○

📘 1. Batch Gradient Descent

Uses **entire dataset** to compute each gradient.

Accuracy: ○ (Good)

- Very stable gradient.
- Converges smoothly.

Time: Slow

- One update per epoch → very slow for large datasets.

Memory Usage: High

- Must load whole dataset at once.

Online Learning: × (Not suitable)

- Cannot update model with one sample at a time.

🟦 2. Stochastic Gradient Descent (SGD)

Uses **only one sample** per update.

Accuracy: Δ (Medium)

- Updates are noisy.
- May oscillate around minimum.
- Final solution not as stable as batch GD.

Time: High (Fast)

- Many quick updates.
- Works well on huge datasets.

Memory Usage: Low

- Needs only a single sample at a time.

Online Learning: ○ (Good)

- Can update the model immediately when new data arri

✅ 3. Iteration

An **iteration** = **one weight update step.**

For a given batch size:

$$\text{Iterations per epoch} = \frac{\text{Dataset size}}{\text{Batch size}}$$

Examples:

🔵 Illustrative Example

Dataset size = **1000**

Batch size = **100**

Then:

- **Batch** = 100 samples
- **Iterations per epoch** = $1000/100 = 10$
- **Epoch** = when you finish those 10 iterations

If you train for **5 epochs** → total iterations

$$5 \times 10 = 50 \text{ iterations}$$

3. Mini-batch Gradient Descent

Uses **small batches** (e.g., 32, 64).

Accuracy: ○ (Good)

- More stable than SGD.
- Faster than batch GD.
- Best practical performance.

Time: Medium

- Faster than batch, slower than SGD.

Memory Usage: Medium

- Needs to load a small batch at a time.

Online Learning: ○ (Good)

- Can be used in streaming data setups.

✅ 1. Batch

A **batch** is the number of training samples processed **before updating** the model weights.

Examples:

- Batch size = 32 → process 32 samples, then update weights
- Batch size = 1 → (this becomes SGD)
- Batch size = all samples → (this becomes Batch Gradient Descent)

✅ 2. Epoch

An **epoch** means **one full pass** over **the entire dataset.**

Example:

- Dataset has **1000** samples
- Batch size = **100**
- To complete **1 epoch**, the model processes all 1000 samples → 1000 samples seen in total.

⭐ Summary Table (Simplified)

Method	Accuracy	Speed	Memory	Online Learning
Batch GD	High	Slow	High	No
SGD	Medium	Fast	Low	Yes
Mini-batch GD	High	Medium	Medium	Yes

✅ Explanation of the Symbols

- ○ **(circle)** = Good
- △ **(triangle)** = Medium / Depends
- × **(cross)** = Not suitable
- ◯ **(hollow circle)** = Suitable

Batch gradient descent

This method computes the gradient of the cost function with **the entire training dataset**.

Update equation

$$\theta = \theta - \eta * \nabla_\theta J(\theta)$$

We need to calculate the gradients for the whole dataset to perform **just one update.**

Code

```
for i in range(nb_epochs):
  params_grad = evaluate_gradient(loss_function, data, params)
  params = params - learning_rate * params_grad
```

Batch gradient descent

- Advantage
  - It is guaranteed to converge **to the global minimum for convex error surfaces and to a local minimum for non-convex surfaces.**

- Disadvantages
  - It can be **very slow**.
  - It is intractable for datasets that **do not fit in memory**.
  - It **does not allow** us to update our model **online.**

Stochastic gradient descent

This method performs a parameter update for **each** training example $x^{(i)}$ and label $y^{(i)}$.

Update equation

$$\theta = \theta - \eta * \nabla_\theta J(\theta; x^{(i)}; y^{(i)})$$

We need to calculate the gradients for the whole dataset to perform **just one update.**

Code

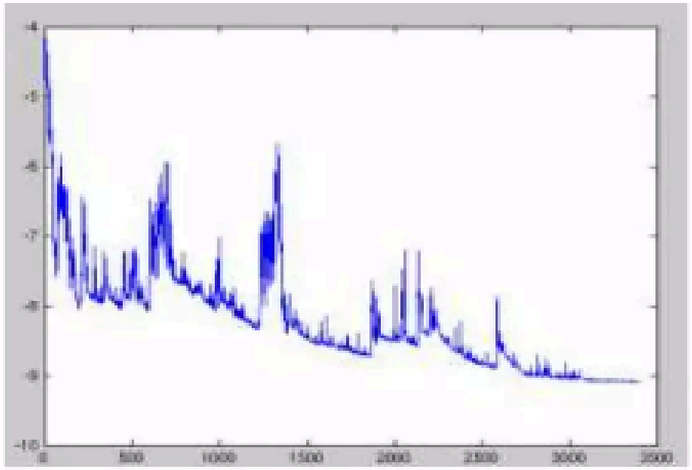
```
for i in range(nb_epochs):
  np.random.shuffle(data)
  for example in data:
    params_grad = evaluate_gradient(loss_function, example, params)
    params = params - learning_rate * params_grad
```

Note : we shuffle the training data at every epoch

Stochastic gradient descent

- Advantage
  - It is usually **much faster** than batch gradient descent.
  - It can be **used to learn online.**

- Disadvantages
  - It performs frequent updates with a **high variance** that cause the objective function to fluctuate heavily.



SGD

We fit a one-parameter linear model (no bias)

$$\hat{y} = wx$$

using squared error loss (per-example):

$$\ell(w; x, y) = \tfrac{1}{2}(wx - y)^2.$$

Dataset (very small): $(x, y) = (1, 2), (2, 4), (3, 6)$ — true slope is 2.

SGD hyperparameters:

- initial weight $w_0 = 0$
- learning rate $\eta = 0.1$
- iterate through samples sequentially (stochastic updates on single examples)

Gradient for one example (x, y):

$$\frac{\partial \ell}{\partial w} = (wx - y)\, x$$

SGD update:

SGD update:

$$w \leftarrow w - \eta \cdot \frac{\partial \ell}{\partial w}$$

Step-by-step numeric updates

Start $w_0 = 0$.

1. **Sample (1,2)**
   - prediction: $\hat{y} = 0$
   - error: $e = 0 - 2 = -2$
   - grad: $g = e \cdot x = -2 \cdot 1 = -2$
   - update: $w_1 = 0 - 0.1 \times (-2) = 0.2$
2. **Sample (2,4)**
   - pred: $\hat{y} = 0.2 \times 2 = 0.4$
   - error: $e = 0.4 - 4 = -3.6$
   - grad: $g = -3.6 \times 2 = -7.2$
   - update: $w_2 = 0.2 - 0.1 \times (-7.2) = 0.9$
3. **Sample (3,6)**

3. **Sample (3,6)**
   - pred: $\hat{y} = 0.92 \times 3 = 2.76$
   - error: $e = 2.76 - 6 = -3.24$
   - grad: $g = -3.24 \times 3 = -9.72$
   - update: $w_3 = 0.92 - 0.1 \times (-9.72) = 1.892$
4. **Sample (1,2)** (next epoch / cycle)
   - pred: $1.892 \times 1 = 1.892$
   - error: -0.108
   - grad: -0.108
   - update: $w_4 = 1.892 - 0.1 \times (-0.108) = 1.9028$
5. **Sample (2,4)**
   - pred: $1.9028 \times 2 = 3.8056$
   - error: -0.1944
   - grad: -0.3888
   - update: $w_5 = 1.9028 - 0.1 \times (-0.3888) = 1.94168$

After five stochastic updates $w \approx 1.9417$, already a[illegible]oaching the true value 2. With more passes (and maybe a slightly smaller learning rate), it will converge very close to 2.

Batch gradient descent

Nice — let's solve the **same toy problem** using **batch (full-batch) gradient descent** (every update uses all training examples).

Problem recap

Model: $\hat{y} = wx$ (no bias)

Dataset: $(1, 2), (2, 4), (3, 6)$ (true line $y = 2x$)

Loss (mean squared, with 1/2 factor):

$$L(w) = \frac{1}{2N} \sum_{i=1}^{N} (wx_i - y_i)^2$$

Gradient (full batch):

$$\nabla L(w) = \frac{1}{N} \sum_{i=1}^{N} (wx_i - y_i)\, x_i$$

Hyperparameters: $w_0 = 0$, learning rate $\eta = 0.1$, $N = 3$.

First (exact) batch update (manual)

Compute per-example contributions at $w_0 = 0$:

- For $(1, 2)$: $(wx - y)x = (0 - 2) \cdot 1 = -2$
- For $(2, 4)$: $(0 - 4) \cdot 2 = -8$
- For $(3, 6)$: $(0 - 6) \cdot 3 = -18$

Sum $= -2 - 8 - 18 = -28$. Gradient:

$$\nabla L(w_0) = \frac{-28}{3} \ (\text{exact})$$

Update:

$$w_1 = w_0 - \eta \nabla L(w_0) = 0 - 0.1 \cdot \left(-\frac{28}{3}\right) = \frac{14}{15} \approx 0.9333333.$$

⭐ Stochastic Gradient Descent (SGD) vs Batch Gradient Descent

Feature	Stochastic Gradient Descent (SGD)	Batch Gradient Descent
Data used per update	Uses **1 sample** at a time	Uses **all samples** of dataset
Update frequency	Updates weights **after every sample**	Updates weights **after full dataset**
Speed	**Fast per update** (very quick steps)	**Slow per update** (heavy computation)
Convergence	Noisy path, may zig-zag	Smooth, stable path
Memory requirement	Very low	Very high (needs whole dataset)
Best for	Large datasets	Small datasets
Chances of getting stuck in local minima	Lower (because of randomness)	Higher
Exactness of gradient	Approximate	Exact true gradient
Training time	Often faster overall	Can be very slow

Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Batch Gradient Descent

- Uses the entire dataset to compute the gradient.
- Stable but slow.
- Good for small datasets.

Stochastic Gradient Descent (SGD)

- Uses one example at a time.
- Very fast, helps escape local minima.
- Path is noisy (zig-zag), but reaches solution faster.
- Good for large datasets (big data).

🎓Example

Dataset has 1000 samples.

Batch Gradient Descent

- Computes gradient using all 1000 samples.
- Updates weights **once**.

SGD

- Computes gradient using 1 sample.
- Updates weights **1000 times**.

Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Mini-Batch Gradient Descent

Mini-batch gradient descent is a combination of Batch GD and SGD.

Instead of using:

- **1 sample (SGD)**
- **all samples (Batch GD)**

it uses a small group of samples, called a mini-batch.

Common batch sizes: 32, 64, 128, 256.

How it works

If dataset = 1000 samples
Batch size = 100

Then each batch = 100 samples
Total batches = 10

The model updates its weights 10 times per epoch.

🔍 Comparison with SGD and Batch

Feature	SGD	Mini-Batch GD	Batch GD
Samples per update	1	32–256 (small chunk)	All samples
Speed per update	Fast	Moderate	Slow
Convergence smoothness	Noisy	Balanced	Smooth
Computation efficiency	Very high	**Highest (best)**	Low
Memory requirement	Very low	Moderate	High
Most commonly used?	No	**YES**	No

✔ Convolutional Neural Networks (CNNs)

- Used for: Image classification, object detection
- Batch size commonly: **32, 64, 128**
- Always uses **Mini-Batch GD**
- Complete dataset (ImageNet) is too big for batch GD
- SGD is too noisy

  → Mini-batch is perfect.

✔ Recurrent Neural Networks (RNNs, LSTMs)

- Used for: speech recognition, text prediction
- Uses **mini-batch (16–64)**
- Faster training & stable gradients.

✔ Transformers (BERT, GPT, Vision Transformers)

- Train on **huge datasets**
- Batch size: **128–2048** (depending on GPU memory)
- Mini-batch helps with parallelization on GPUs (A , V100).

Method	Use Case
Batch GD	Very small datasets, teaching/demo purposes
SGD	Online learning, streaming data
Mini-Batch GD	Almost all deep learning training

Optimizer	Stores	Pros	Cons
Vanilla GD	None	simple	slow, unstable
SGD	None	fast	noisy
Momentum	1st moment	smoother	still fixed LR
RMSProp	2nd moment	adaptive LR	no momentum
Adam	both	fast, stable, adaptive	sometimes over-adapts

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Recurrent Neural Networks

Introduction

- In traditional **neural networks**, inputs and outputs are treated independently.
- However, tasks like predicting the next word in a sentence require information from previous words to make accurate predictions.
- To address this limitation, *Recurrent Neural Networks (RNNs)* were developed.

Introduction

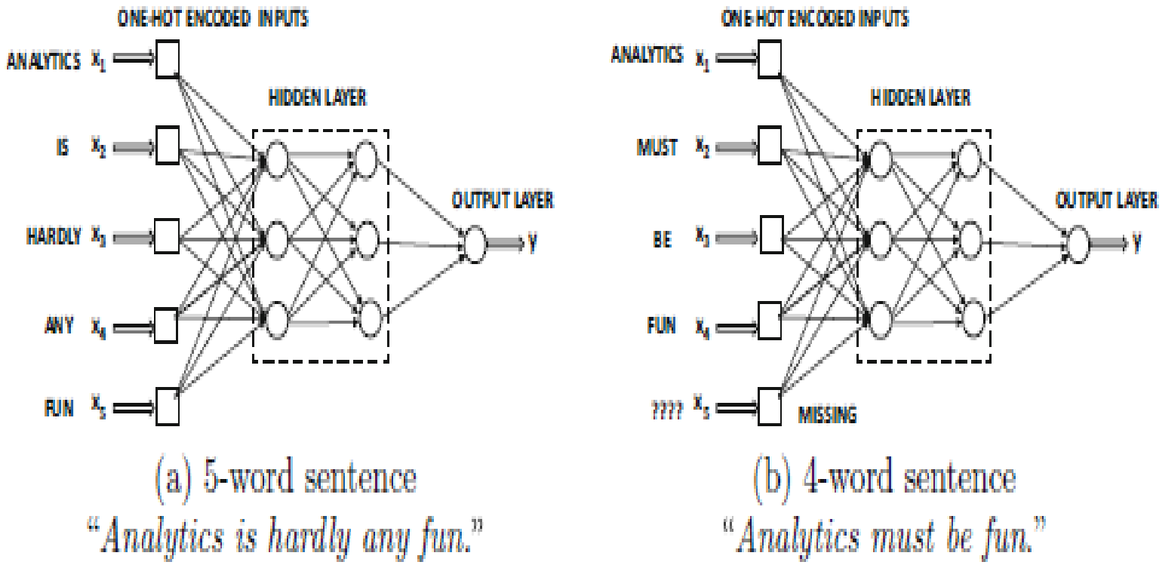
- Certain data types such as time-series, text, and biological data contain sequential dependencies among the attributes.
- In a time-series data set, the values on successive time-stamps are closely related to one another.
- If one uses the values of these time-stamps as independent features, then key information about the relationships among the values of these time-stamps is lost.
- Text is often processed as a bag of words, one can obtain better semantic insights when the ordering of the words is used.
- In such cases, it is important to construct models that take the sequencing information into account.The individual values in a sequence can be either *real-valued or symbolic*.
- Recurrent neural networks can be used for either type of data.

Challenges of Traditional NN

Bag of Words Approach:

- In some applications, text is processed by treating it as a "bag of words." This means the words in the text are considered without caring about their order. For example, "The cat chased the mouse" and "The mouse chased the cat" would be treated the same because they contain the same words.
- When Order Matters:
- This method works fine when the text is long (like a large document) because the overall meaning can often still be understood.
- However, in cases where word order is crucial—like in short text or sentences—this approach fails. The sequence of words changes the meaning. For instance:
- "The cat chased the mouse" means the cat is the one doing the chasing.
- "The mouse chased the cat" completely flips the meaning, making the mouse the one chasing.

Challenges of Traditional NN



(a) 5-word sentence
"Analytics is hardly any fun."

(b) 4-word sentence
"Analytics must be fun."

One possible solution is to avoid the bag-of-words approach and create one input for each position in the sequence.

Consider a situation in which one tried to use a conventional neural network in order to perform sentiment analysis on sentences with one input for each position in the sentence.

The sentiment can be a binary label depending on whether it is positive or negative.

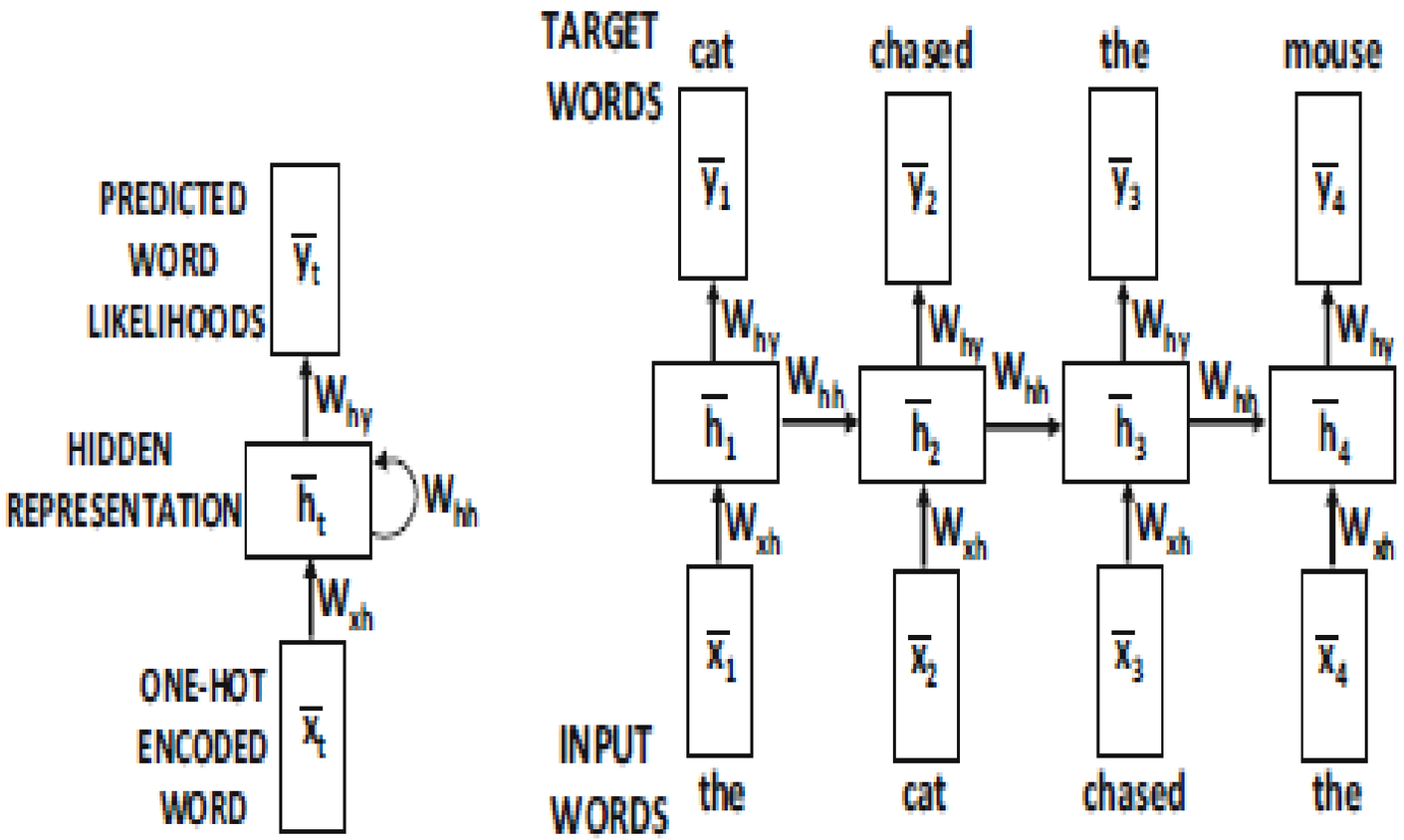
The first problem that one would face is that the length of different sentences is different

Architecture of RNN

- Recurrent Neural Networks introduce a mechanism **where the output from one step is fed back as input to the next, allowing them to retain information from previous inputs.**

- **This design makes RNNs well-suited for tasks where context from earlier steps is essential, such as predicting the next word in a sentence**

Architecture of RNN



(a) RNN (b) Time-layered representation of (a)

Updating the Hidden State in RNNs

1. State Update:

$$h_t = f(h_{t-1}, x_t)$$

where:

- h_t is the current state
- h_{t-1} is the previous state
- x_t is the input at the current time step

2. Activation Function Application:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

Here, W_{hh} is the weight matrix for the recurrent neuron, and W_{xh} is the weight matrix for the input neuron.

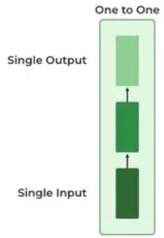
3. Output Calculation:

$$y_t = W_{hy} \cdot h_t$$

where y_t is the output and W_{hy} is the weight at the output layer.

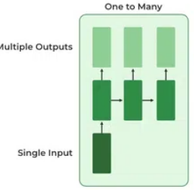
Types of RNN

- **One-to-One RNN** : In this setup, there is a single input and a single output.
- Commonly used for straightforward classification tasks where input data points do not depend on previous elements.



One to One RNN

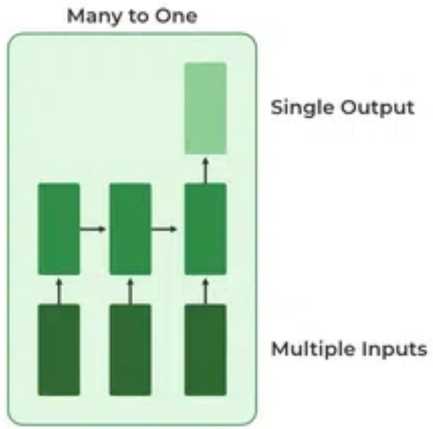
- **One-to-Many RNN**
- In a **One-to-Many RNN**, the network processes a single input to produce multiple outputs over time.
- This setup is beneficial when a single input element should generate a sequence of predictions.
- For example, for image captioning task, a single image as input, the model predicts a sequence of words as a caption.



One to Many RNN

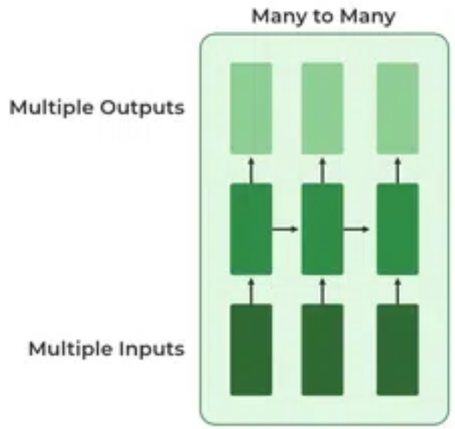
Types of RNN

- **Many-to-One RNN:** The **Many-to-One RNN** receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction.
- **In sentiment analysis, the model receives a sequence of words (like a sentence) and produces a single output, which is the sentiment of the sentence (positive, negative, or neutral).**



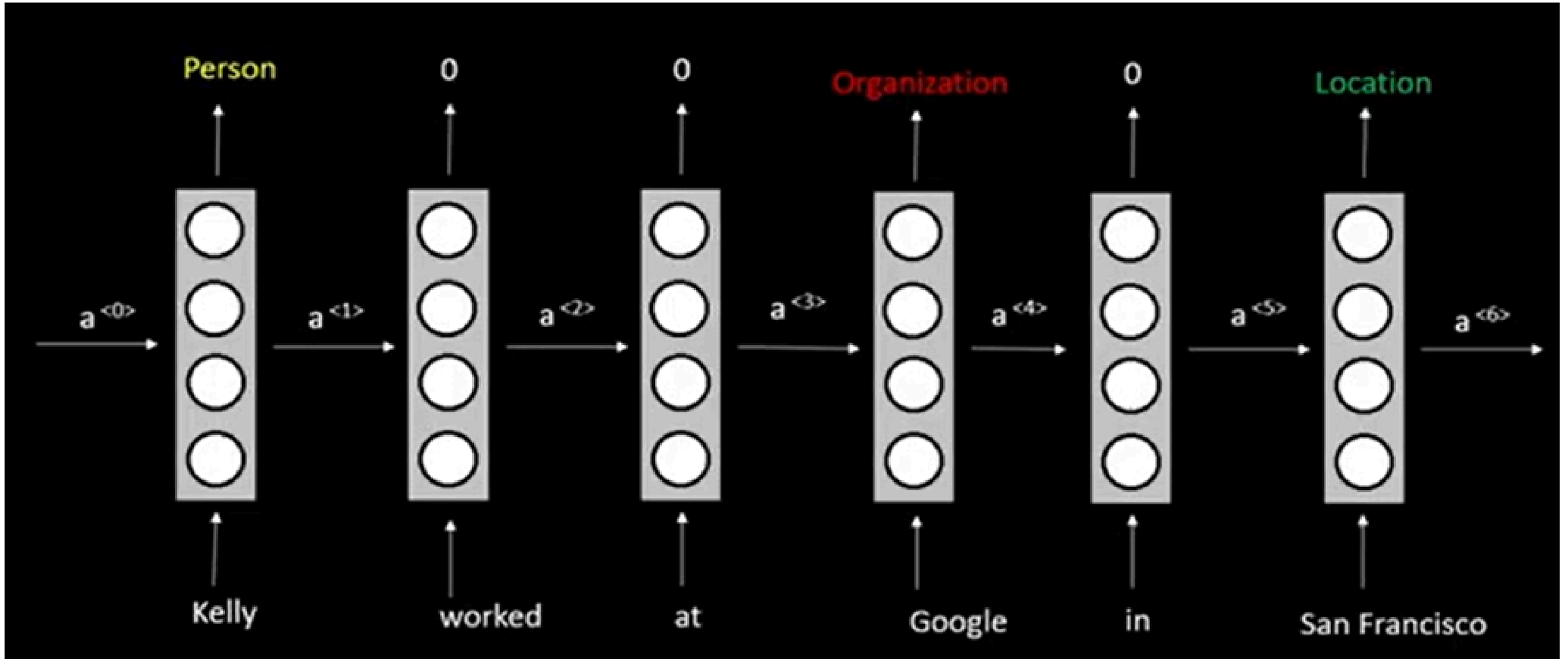
Types of RNN

- **Many-to-Many RNN:** The **Many-to-Many RNN** type processes a sequence of inputs and generates a sequence of outputs.
- This configuration is ideal for tasks where the input and output sequences need to align over time, often in a one-to-one or many-to-many mapping.
- In language translation task, a sequence of words in one language is given as input, and a corresponding sequence in another language is generated as output.



Example -2

- Identify the person, organization and Location (Named Entity Recognition)
  - **Kelly worked at google in San Francisco**



A bidirectional RNN processes sequence data in both forward and backward directions.
It uses two RNNs: one that reads the sequence from start to end (forward), and another that reads it from end to start (backward).

- Architecture:
  - Each timestep in the sequence has two hidden states:
    - $\overrightarrow{h_t}$: The forward hidden state.
    - $\overleftarrow{h_t}$: The backward hidden state.
  - The output at each timestep combines these two hidden states (e.g., concatenation or summation).

Bi-Directional RNN

- **Forward Pass:**
  - The forward RNN computes:
$$\overrightarrow{h_t} = \sigma(W_{xh}x_t + W_{hh}\overrightarrow{h_{t-1}})$$
  - The backward RNN computes:
$$\overleftarrow{h_t} = \sigma(W_{xh}x_t + W_{hh}\overleftarrow{h_{t+1}})$$
- **Final Output:**
  - At each timestep t, the combined output is:
$$y_t = f(\overrightarrow{h_t}, \overleftarrow{h_t})$$
    where f is typically concatenation or another aggregation function.
- **Advantages:**
  - **Context Awareness:** Both past and future contexts are considered at each timestep.
  - **Improved Accuracy:** Especially useful in tasks where the entire sequence matters (e.g., speech recognition, language translation).

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Convolutional Neural Network

Introduction

- A **Convolutional Neural Network (CNN)** is a type of Deep Learning neural network architecture commonly used in Computer Vision.

- **Why CNN?**

Key Limitations of Traditional Neural Networks:

- **High Computational Cost**
- **Loss of Spatial Information**
- **Inefficient for Large Images**

Example: Handwritten Digit Recognition

- **Task: Classify a handwritten digit from a grayscale of 28× 28-pixel image.**

Using a Traditional Neural Network:

Input size: 28× 28=784 pixels. Each pixel is treated as an independent feature.

Number of parameters (1 hidden layer, 1000 neurons): 784× 1000=784,000.

This network ignores spatial patterns (e.g., edges or curves forming digits) and relies on brute-force learning, leading to poor efficiency and generalization.

Using a CNN:

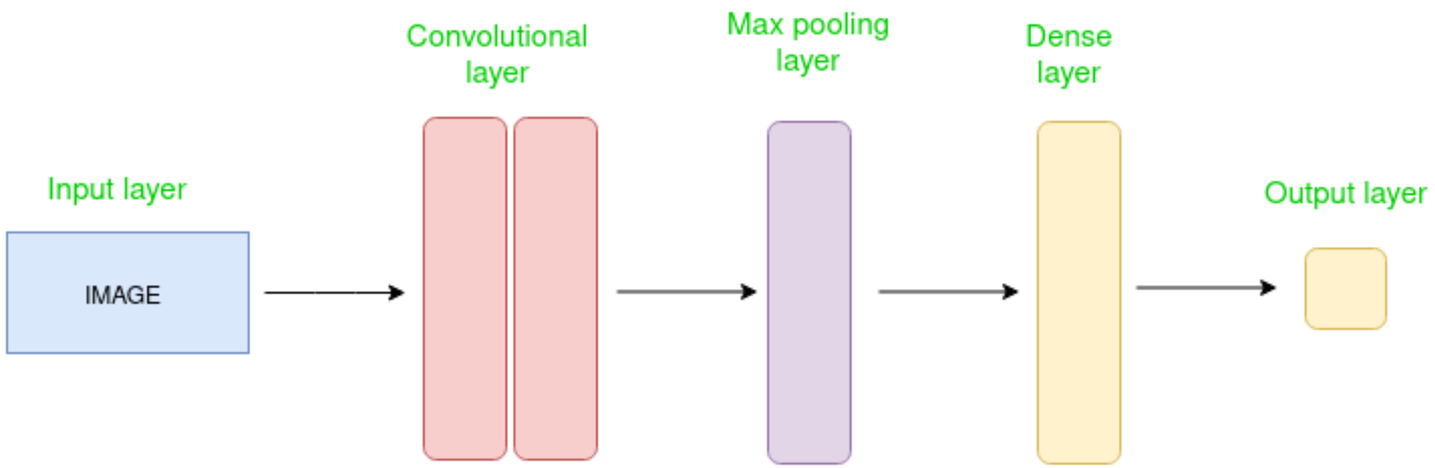
Instead of feeding all pixels at once, a convolutional layer applies filters (e.g., 3× 3 to detect edges, curves, and textures.

A pooling layer (e.g., max pooling) reduces dimensions while preserving key features.

The network learns hierarchical patterns: edges → shapes → digits.

Number of parameters for a filter: 3× 3=9 . Even with multiple filters, parameter count is far smaller than an FCNN.

Basic Structure of a Convolutional Network



- The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction.

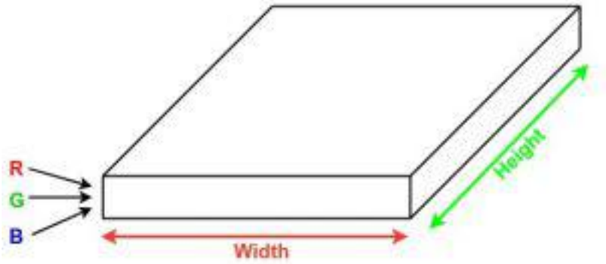
- The network learns the optimal filters through backpropagation and gradient descent

Convolutional Layer (Filter)

A **filter** (also called a **kernel**) in a Convolutional Neural Network (CNN) is a small matrix of learnable weights used to detect specific features in input data, such as **edges, textures**, or **patterns.**

Filters are at the heart of the convolutional operation, **enabling CNNs to learn and extract hierarchical features from input images.**

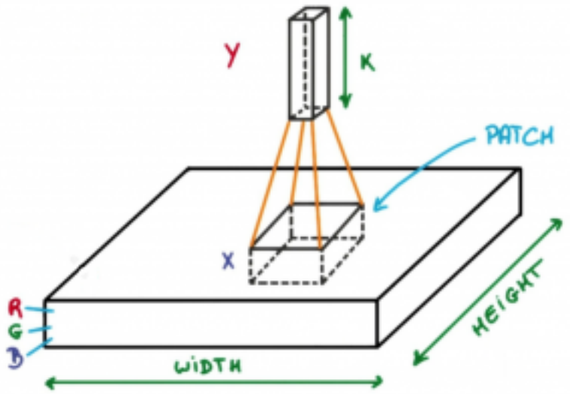
Size:
Filters are typically much smaller than the input data. Common sizes include 3×3 5×5, 7×7
- Example: For a 28×28, 3×3 filter slides across the image to analyze local regions.



Now imagine taking a small patch of this image and running a small neural network, called a filter or kernel on it, with say, K outputs and representing them vertically.

Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths.

Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called Convolution



1	6	9	10	2	8
2	5	1	8	4	2
3	7	4	9	10	3
9	8	3	6	7	9
8	0	9	4	7	2
9	10	12	6	9	8

*

1	0	-1
1	0	-1
1	0	-1

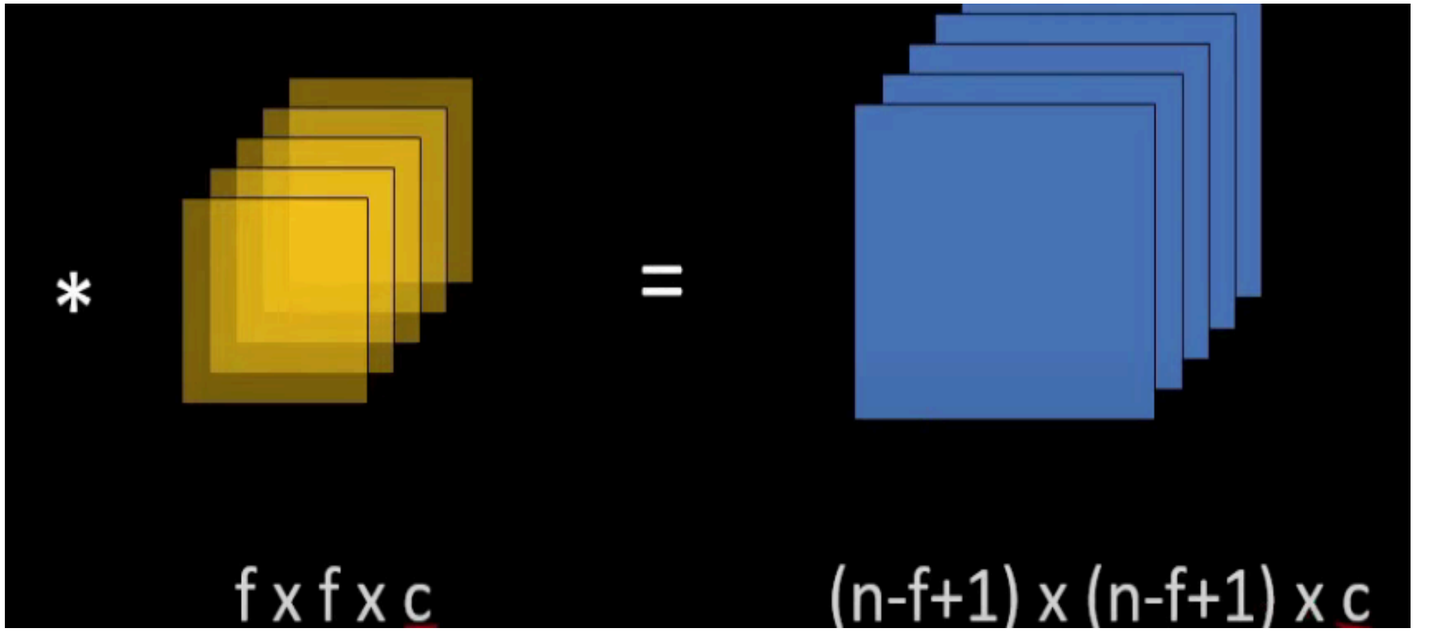
=

11	6 0	9 -1	10	2	8
21	5 0	1 -1	8	4	2
31	7 0	4 -1	9	10	3
9	8	3	6	7	9
8	0	9	4	7	2
9	10	12	6	9	8

-8	-9	-2	14
6	-3	-13	9
4	-4	-8	5
2	2	1	-3

$$(n \times n) * (f \times f) = (n - f + 1) \times (n - f + 1)$$

1	6	9	10	2	8
2	5	1	8	4	2
3	7	4	9	10	3
9	8	3	6	7	9
8	0	9	4	7	2
9	10	12	6	9	8





*

1	0	-1
1	0	-1
1	0	-1

=



Vertical Edges

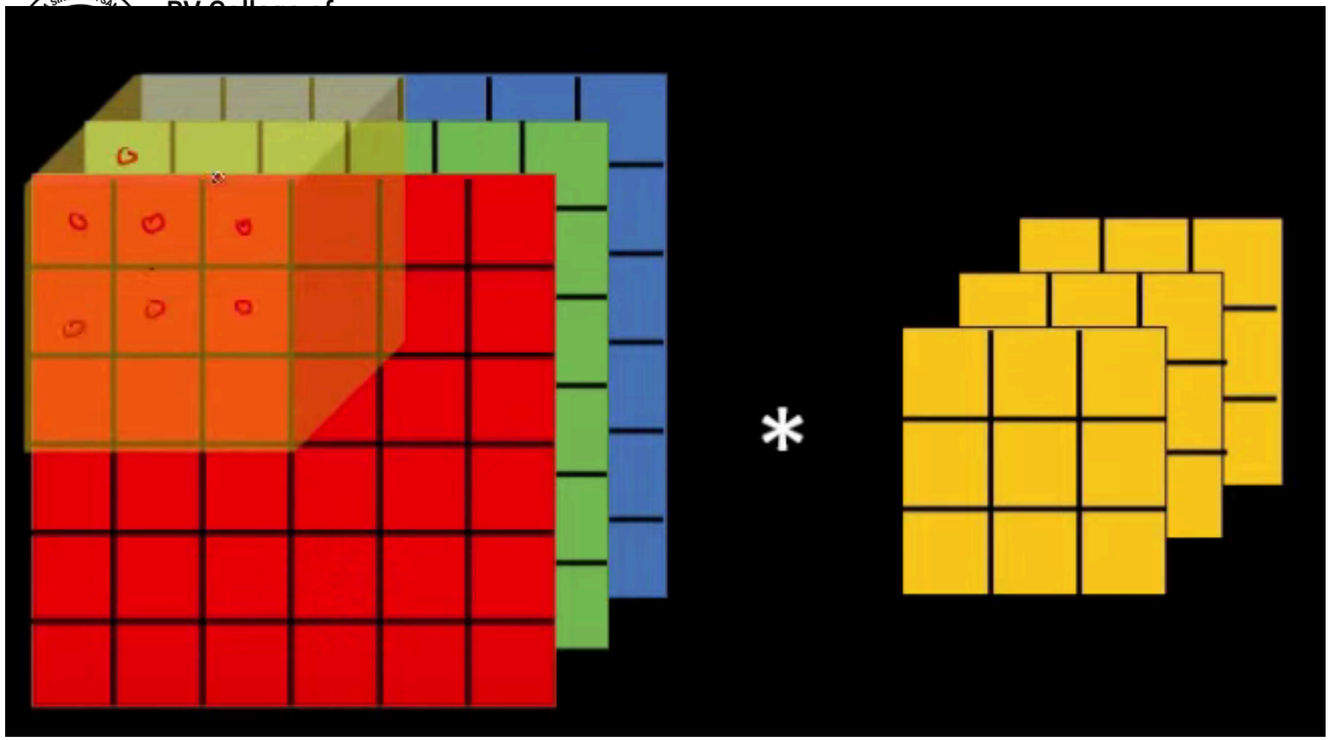


*

1	1	1
0	0	0
-1	-1	-1

=



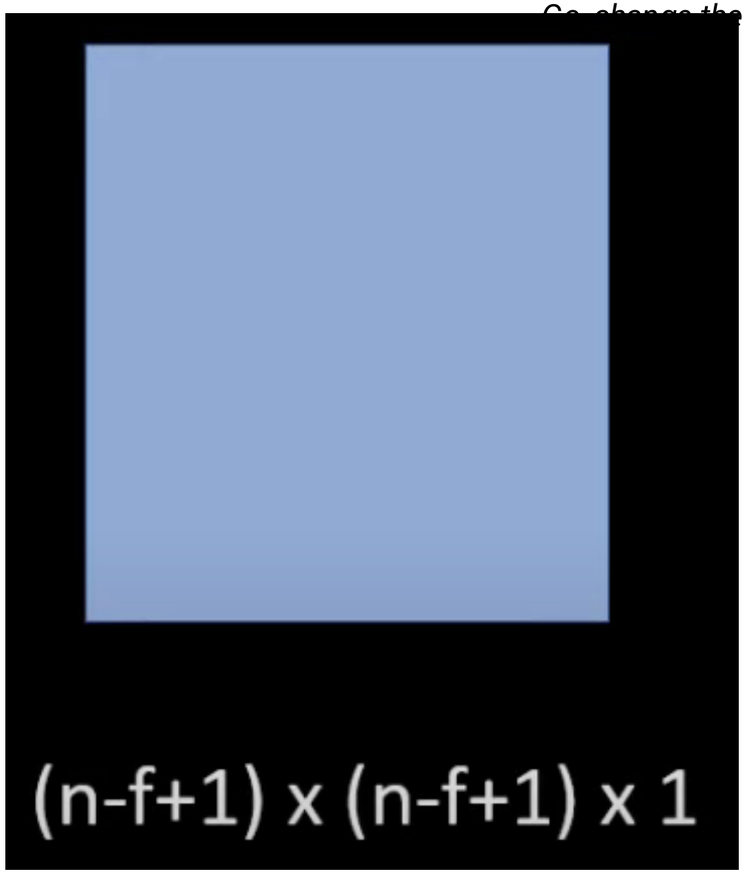


n x n x 3

*

f x f x 3

=



(n-f+1) x (n-f+1) x 1

- **Stride** in a Convolutional Neural Network (CNN) refers to the step size with which the convolutional filter moves across the input feature map during the convolution operation.

Key Characteristics of Stride

1. **Default Stride:**
   1. A stride of S=1 means the filter moves one pixel at a time.
   2. This results in maximum overlap between adjacent filter applications and produces the largest output size.
2. **Larger Stride:**
   1. A stride of S=2means the filter moves two pixels at a time.
   2. This skips some input data, resulting in a smaller output feature map (down sampling).
   3. S = [n-f/s]+1

-

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Padding

Padding in Convolutional Neural Networks (CNNs) refers to adding extra layers of pixels around the edges of an input image or feature map. This technique is used to control the spatial dimensions of the output after a convolution operation.

Why Use Padding?

1. Preserve Spatial Dimensions:

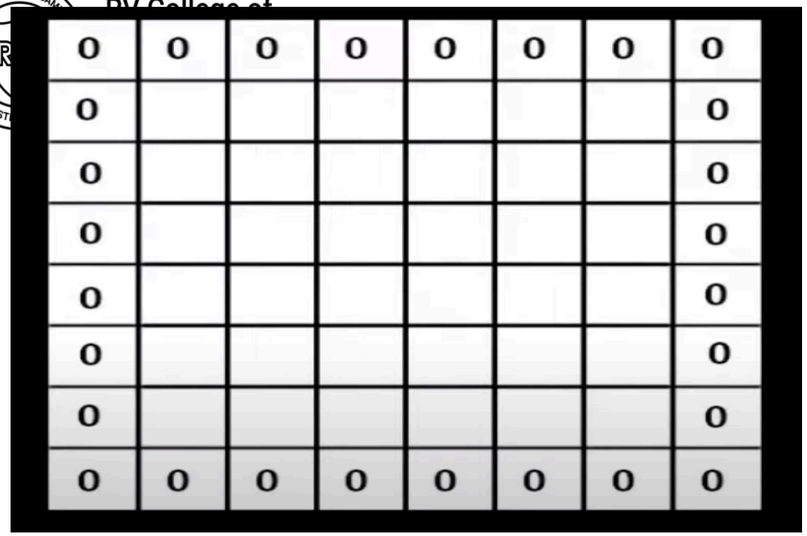
1. Without padding, applying filters reduces the dimensions of the input feature map, especially near the edges.
2. Padding ensures that the output has the same dimensions as the input (in "same" padding).

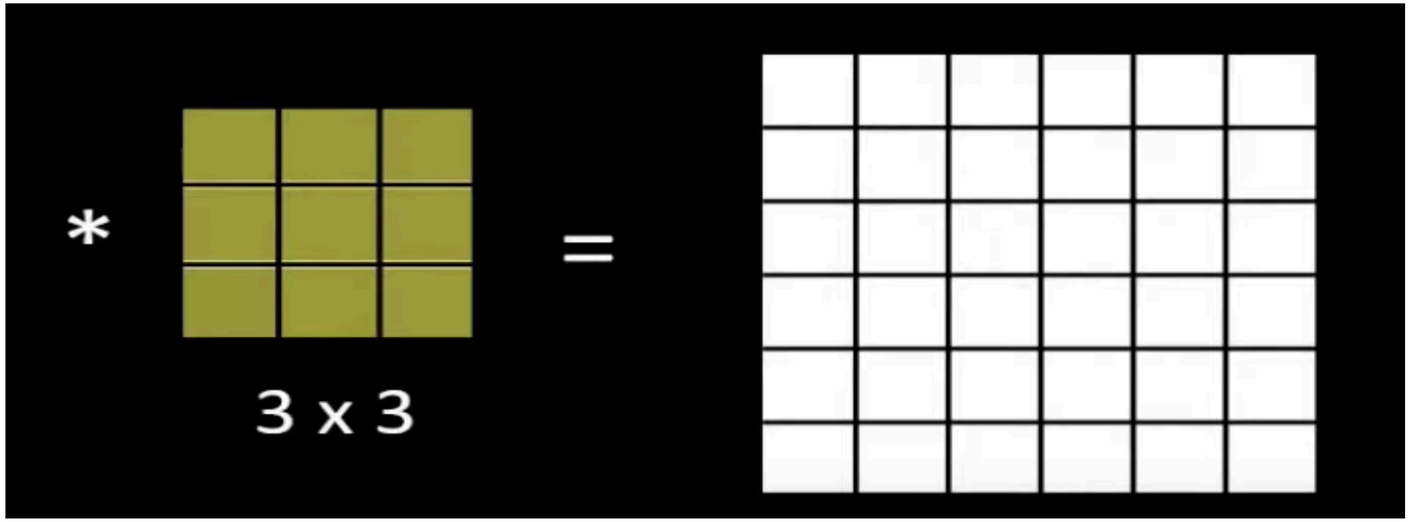
2. Prevent Information Loss:

1. Filters inherently struggle to process the edges of an image because there are fewer surrounding pixels.
2. Padding provides artificial pixels (usually zeroes) so that edge features are captured effectively.

3. Allow Deeper Networks:

1. Maintaining dimensions across layers (using padding) helps design deeper networks without quickly shrinking the feature maps.





Types of Padding

1.Zero Padding:

1. Adds zero-value pixels around the edges.
2. Most common type of padding used in CNNs.

2.Valid Padding:

1. No padding is added. Convolutions are applied only where the filter completely overlaps the input.
2. Results in smaller output dimensions.

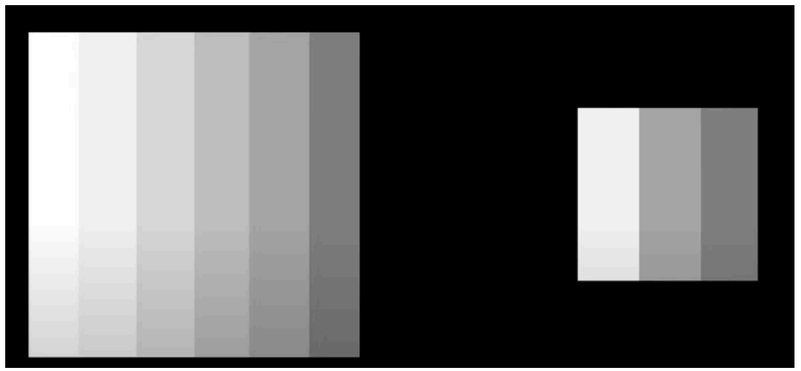
3.Same Padding:

1. Adds enough padding so the output dimensions are the same as the input dimensions.
2. Ensures consistent size across layers.

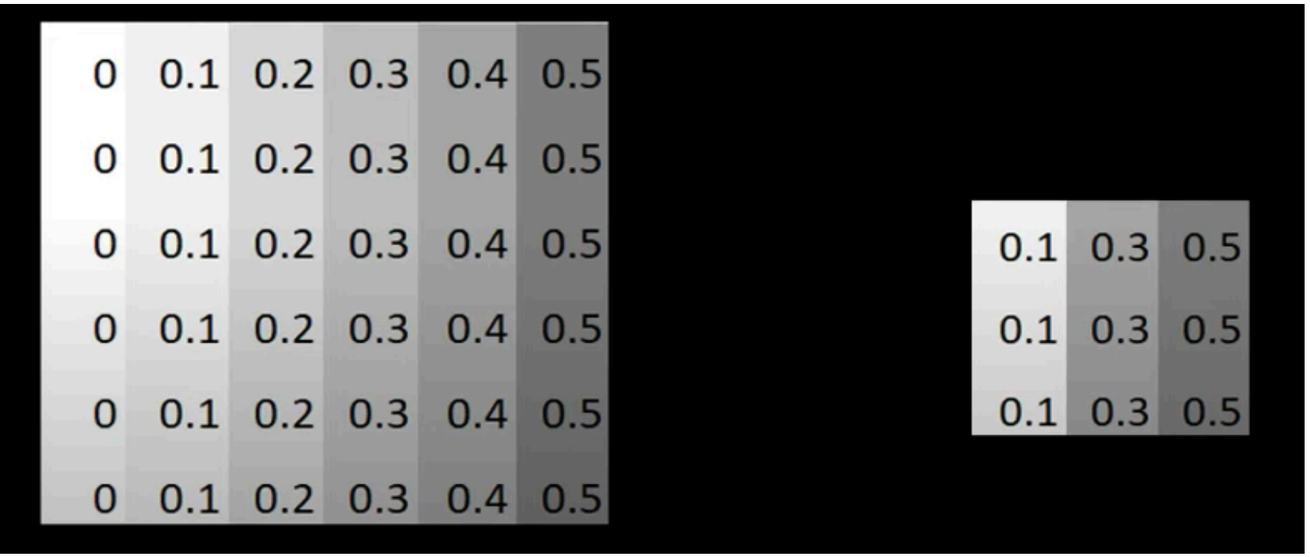
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Pooling

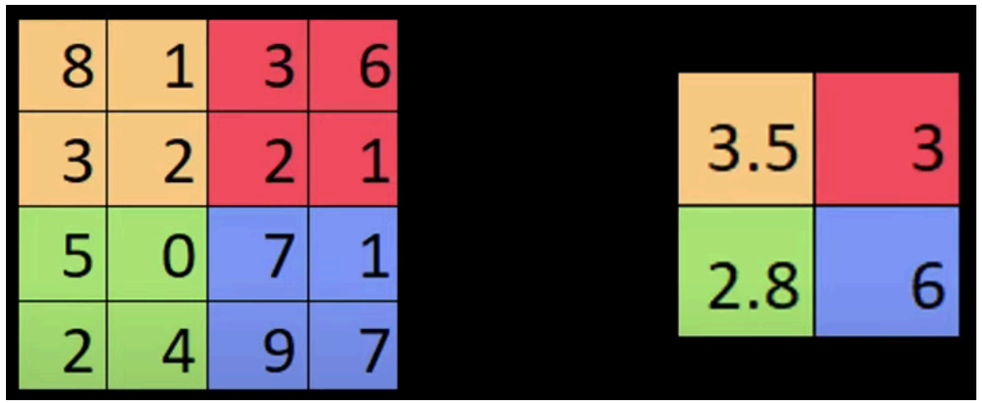
- **Pooling in Convolutional Neural Networks (CNNs)** is a technique used to reduce the spatial dimensions of the input feature maps while retaining essential features.
- **It helps in reducing the computational complexity and controlling overfitting by summarizing the features in a region of the image.**
- **Pooling is typically applied after the convolutional layers to reduce the size of the feature maps and make the network more computationally efficient.**
- **There are two type of pooling : Max Pooling and Average Pooling**



Max Pooling



Average Pooling



Typical Settings

- **Filter Size (Kernel Size):**
  - Commonly 3x3 or 5x5 for convolution operations.
  - Determines the receptive field to extract features like edges and textures.
- **Stride:**
  - Default is 1 for overlapping coverage.
  - Larger strides (e.g., 2) reduce dimensionality but risk losing fine-grained details.
- **Padding:**
  - Same Padding: Maintains input dimensions by padding with zeros.
  - Valid Padding: No padding, reduces spatial dimensions.
- **Pooling Layer:**
  - Typically uses max pooling or average pooling with a size of 2x2 and a stride of 2.
  - Reduces spatial dimensions and retains dominant features.

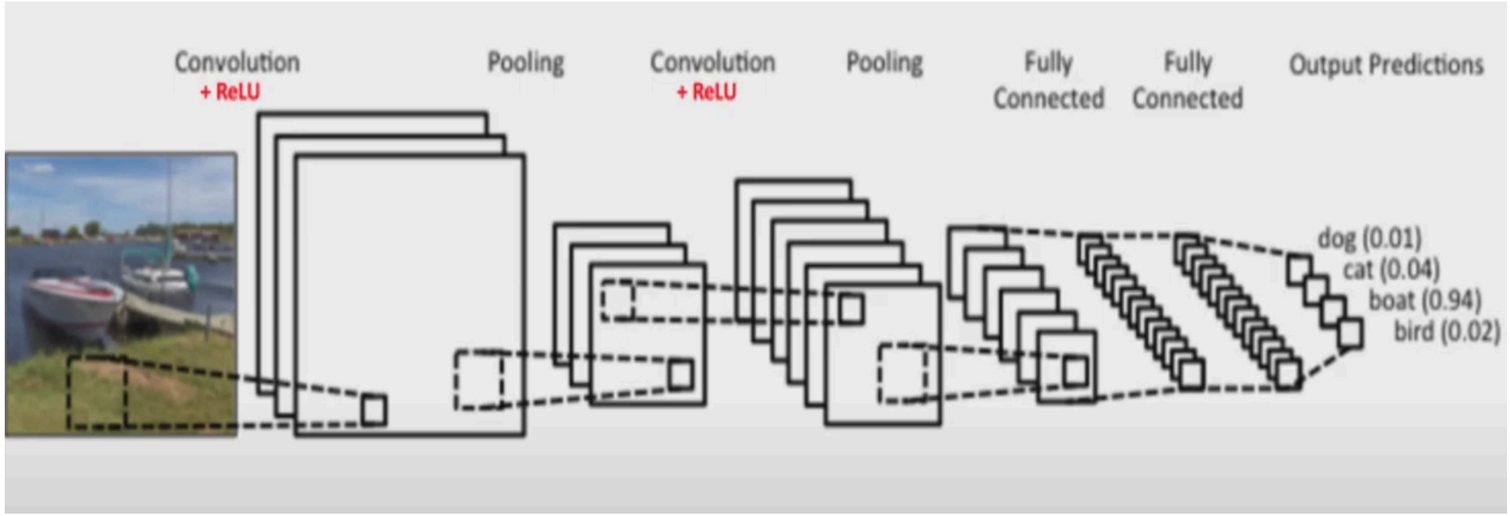
Typical Settings

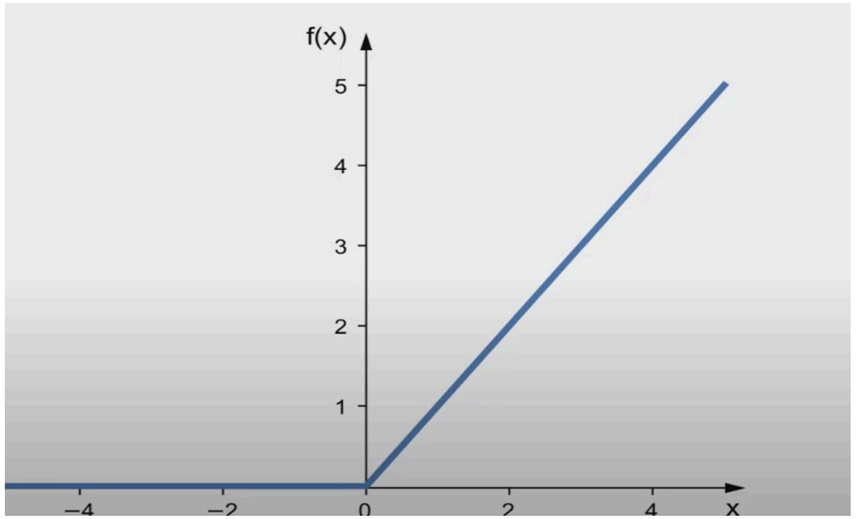
- **Activation Functions:**
  - ReLU (Rectified Linear Unit) is widely used for introducing non-linearity.
  - Alternatives include Leaky ReLU, Tanh, or Sigmoid depending on the task.
- **Number of Filters (Channels):**
  - Starts with fewer filters (e.g., 32 or 64) in early layers.
  - Increases in deeper layers (e.g., 128, 256, 512) to capture complex patterns.
- **Fully Connected Layers:**
  - Typically 1–2 layers at the end, used for classification or regression tasks.

The ReLU layer

- The ReLU (Rectified Linear Unit) layer is a key component in convolutional neural networks (CNNs) that helps introduce non-linearity to the model.

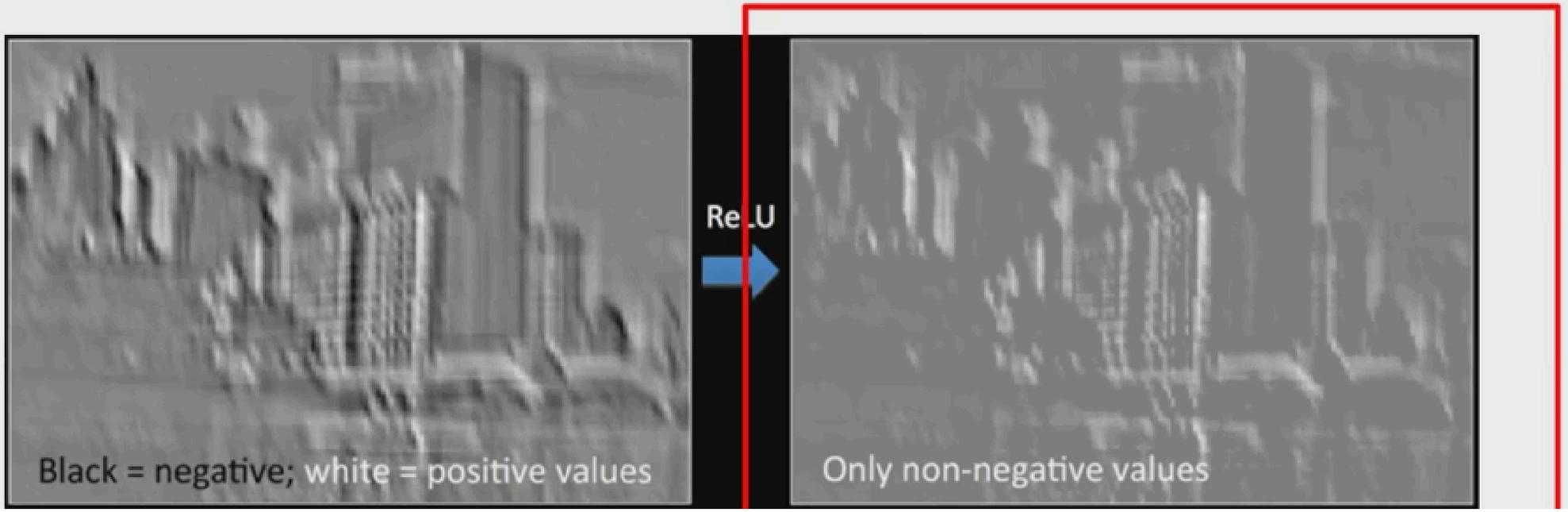
The ReLU activation function is defined as f(x)=max(0,x) This means that if the input is positive, it passes through unchanged, but if the input is negative, it is set to zero. This introduces non-linearity into the network, allowing it to learn complex patterns.





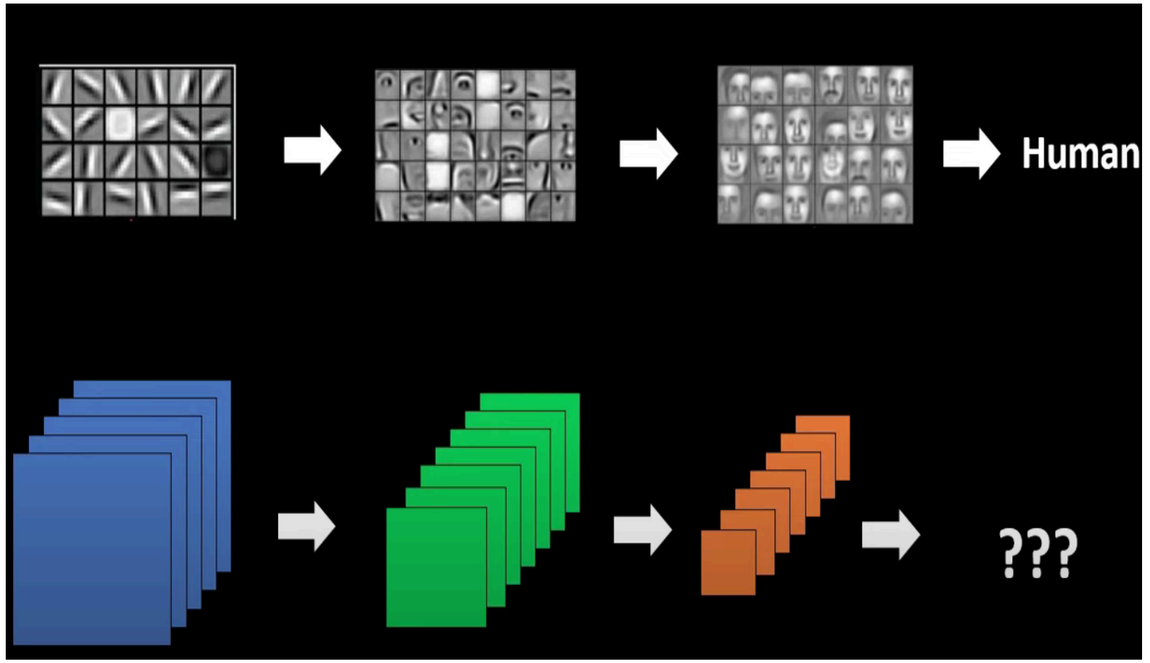
The ReLU Layer





- After extracting all the features, what next?

- How do I classify or label?

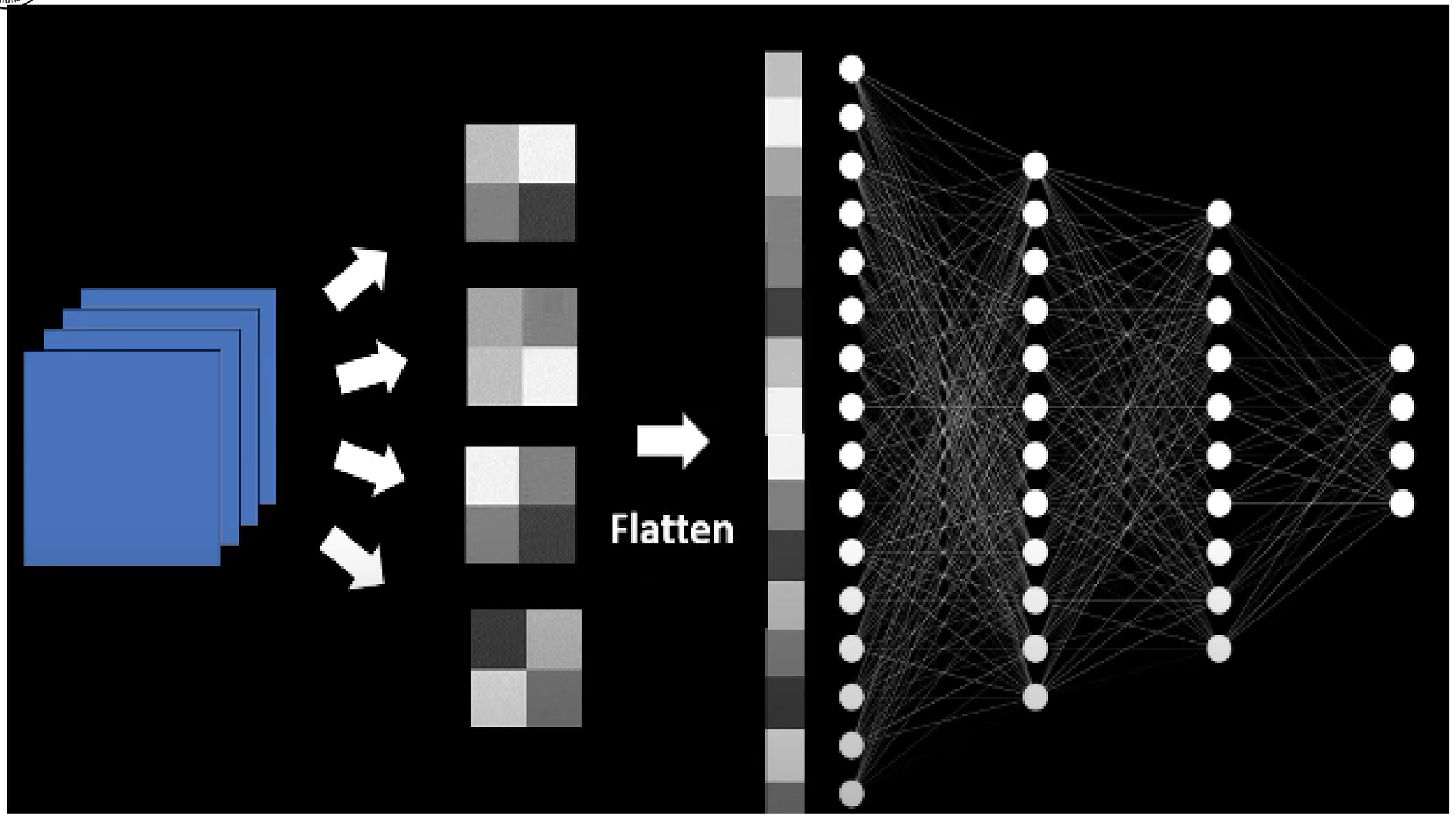


[Input Image] → [Convolutional Layers] → [Pooling Layers] → [Fully Connected Layers] → [Output]

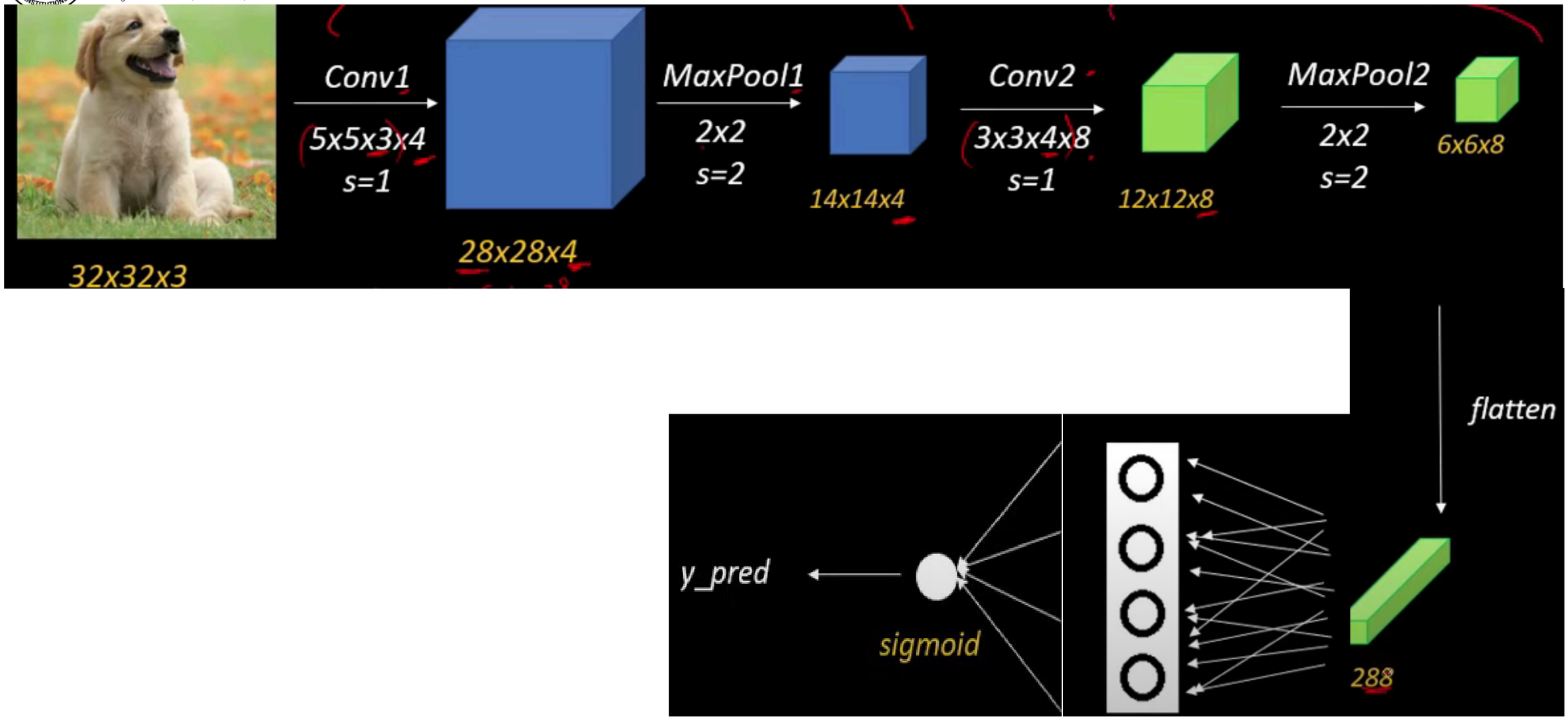
Fully Connected Layer

- Fully connected layers are dense network of neurons.
- Applied after convolutional and max pooling layers.
- Classifies the output
- Associate features to a particular label

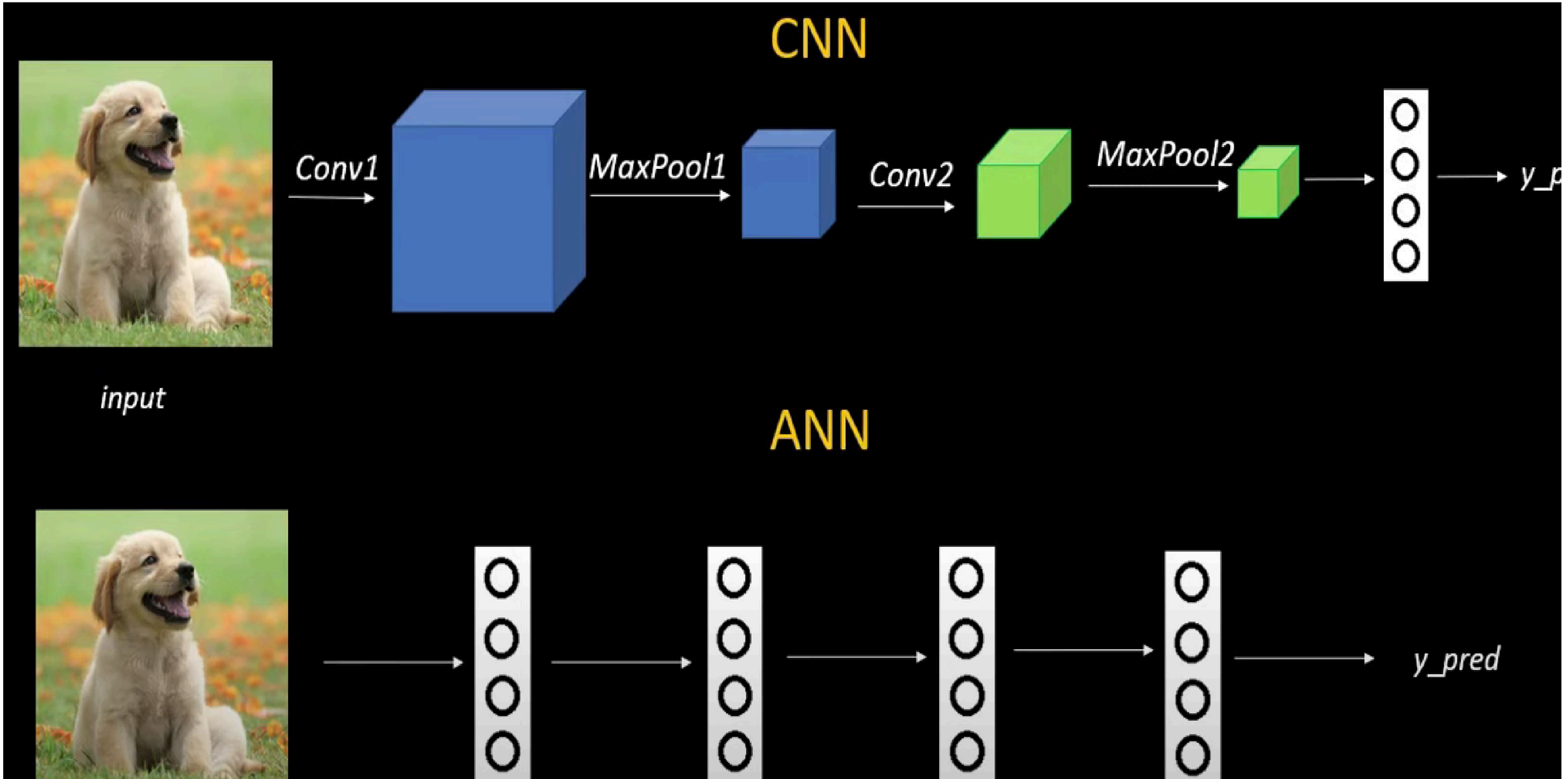
Fully Connected Layer



RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India



Comparison between CNN and ANN



RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Data Augmentation

- **What is Data Augmentation?**
  - It's a technique to reduce overfitting by generating new training data from existing data using transformations.
- **Why is it Useful?**
  - Increases the generalization ability of the model.
  - Helps the model recognize objects in different orientations, lighting, and conditions.
- **Common Transformations in Image Processing:**
  - **Translation:** Shift the image slightly in any direction.
  - **Rotation:** Rotate the image by a certain angle.
  - **Reflection (Mirroring):** Flip the image horizontally or vertically.
  - **Patch Extraction:** Use smaller patches or crops from the image.
  - **Color Intensity Adjustment:** Vary the brightness or color levels.

Reinforcement Learning (RL) and Applications

- Reinforcement Learning (RL)
  - Comprehensive Overview and Applications

Introduction to Reinforcement Learning

- Reinforcement Learning (RL) is a machine learning paradigm where an agent learns optimal behavior through interactions with an environment.
  - Unlike supervised learning, RL does not use labeled input/output pairs. Instead, learning is driven by rewards and penalties.
  - RL is inspired by behavioral psychology and trial- and- error mechanisms.

Core Components of RL

- Agent: The learner or decision-maker that interacts with the environment.
  - Environment: Everything the agent interacts with and receives feedback from.
  - State: A representation of the current situation of the environment.
  - Action: A step or choice taken by the agent.
  - Reward: Feedback from the environment indicating the success of an action.
  - Episode: A full sequence of states, actions, and rewards from start to terminal state.

The RL Process (Feedback Loop)

- 1. The agent observes the current state of the environment.
  - 2. It chooses an action based on a policy.
  - 3. The environment transitions to a new state and provides a reward.
  - 4. The agent updates its knowledge to improve future actions.
  - 5. This loop continues until the task is completed.
  - Goal: Maximize cumulative long- term reward.

Policies, Value Functions, and Models

- Policy (π): Defines the agent’ s behavior by mapping states to actions.
  - Value Function: Estimates how good a state or action is in terms of expected future rewards.
  - Model: Predicts state transitions and rewards; used in model- based RL.
  - Model- free RL: Learns directly from interactions without predicting future states.

Types of Reinforcement Learning Algorithms

- Model-Free Methods: Learn actions without modeling environment transitions.
  - Examples: Q-Learning, SARSA, Deep Q-Network (DQN).
  - Model-Based Methods: Use predictive models for planning.
  - Policy Gradient Methods: Learn policies directly rather than value functions.
  - Actor-Critic Methods: Combine value-based and policy- based strategies for stability and efficiency.

Q-Learning in Detail

- Q-Learning is an off-policy model-free RL algorithm.
  - It learns the value (Q-value) of taking an action in a given state.
  - Q-value Update Formula: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$
  - α = learning rate, γ = discount factor for future rewards.
  - Goal: Learn the optimal action-selection strategy for each state.

Deep Reinforcement Learning (Deep RL)

- Combines RL with deep neural networks to handle high-dimensional input spaces.
  - DQN (Deep Q-Network): Uses CNNs to learn policies directly from pixels (e.g., Atari games).
  - Advanced Deep RL algorithms: A3C, PPO, DDPG, SAC.
  - Applications: Robotics, autonomous driving, games, energy optimization, etc.

Exploration vs. Exploitation

- Exploration: Trying new actions to discover potentially better rewards.
  - Exploitation: Leveraging known actions that provide high rewards.
  - Balancing both is critical for effective learning.
  - Popular strategies: ε-greedy, softmax (Boltzmann) exploration, entropy-regularized policies.

Applications in Robotics

- Robot control and motion planning.
  - Manipulation tasks such as grasping and object sorting.
  - Human-robot collaboration and behavior adaptation.
  - Navigation in uncertain and dynamic environments.
  - Example: RL robots learning to walk, balance, or climb.

Applications in Games and Simulations

- RL has achieved superhuman performance in many games.
  - DeepMind's AlphaGo, AlphaZero, and MuZero.
  - Applications include strategic planning, multi- agent simulations, and adversarial training.
  - Used in e-sports AI, Atari games, and chess engines.

Applications in Autonomous Vehicles

- Decision-making for lane changes, braking, and speed control.
  - Path planning in dynamic environments with pedestrians and obstacles.
  - RL helps autonomous vehicles adapt to complex real- world uncertainties.
  - Used in simulation environments before real-world deployment.

Applications in Finance

- Portfolio management using long-term reward optimization.
  - Algorithmic trading strategies based on sequential decision-making.
  - Risk management and hedging strategies using RL agents.
  - Market simulation for training robust trading policies.

Applications in Healthcare

- Personalized treatment planning based on patient-specific conditions.
  - Optimizing drug dosage schedules.
  - Robotic surgery assistance and autonomous decision-making.
  - Drug discovery and molecule interaction simulations.

Challenges and Limitations

- Sample inefficiency: Requires large amounts of training data.
  - Sparse or delayed rewards hinder learning.
  - High computational cost for deep RL models.
  - Difficulty in ensuring safety, reliability, and interpretability.
  - Challenges in transferring simulation-trained models to the real world.

Future Directions in RL

- Safe RL: Ensuring predictable and safe behavior.
  - Explainable RL: Making agent decisions interpretable.
  - Multi-Agent RL: Training multiple agents that interact and cooperate.
  - Hybrid RL with symbolic AI and large language models.
  - Scaling RL for real- world impactful applications.